Lec08：行为型模式

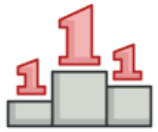
Behavioral Design Patterns

课程编号：22WHSE22006（2023版培养计划）
授课对象：23 级软件工程、服务科学与工程专业
主讲教师：辛国栋
Email：xinguodong@163.com

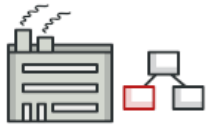


QQ 群：软件构造2025
群　号：311528424

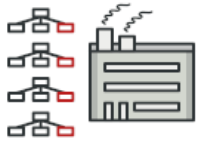
创建型模式(5 种)
Creational Patterns



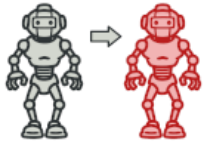
单例
Singleton



工厂方法
Factory Method



抽象工厂
Abstract Factory



原型
Prototype



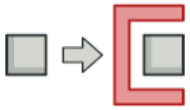
建造者
Builder



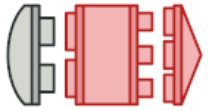
编外：简单工厂
Simple Factory

对象产生与使用解耦

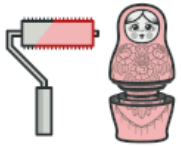
结构型模式(7 种)
Structural Patterns



代理
Proxy



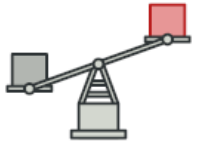
适配器
Adapter



装饰
Decorator



桥接
Bridge



享元
Flyweight



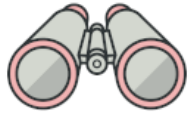
组合
Composite



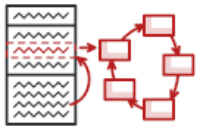
外观
Facade

功能解耦

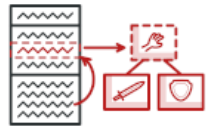
行为型模式 (11 种)
Behavioral Patterns



观察者
Observer



状态
State



策略
Strategy



模版方法
Template Method



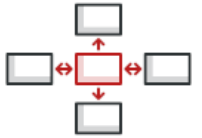
访问者
Visitor



解释器
Interpreter



备忘录
Memento



中介
Mediator



迭代器
Iterator

行为解耦



责任链
Chain of Responsibility



命令
Command

引言：十一种行为型模式的作用

No	行为型模式	作用
1	观察者 Observer	维护多个观察者依赖，状态变化通知所有观察者
2	模板方法 Template Method	父类定义算法骨架，某些实现放在子类
3	策略 Strategy	每种算法独立封装，根据不同情况使用不同算法策略
4	职责链 Chain of Responsibility	所有处理者封装为链式结构，依次调用
5	状态 State	每种状态独立封装，不同状态内部封装了不同行为
6	中介者Mediator	取消类/对象的直接调用关系，使用中介者维护
7	命令 Command	将一个请求封装为一个对象，使发出请求的责任和执行请求的责任分割开
8	备忘录 Memento	把核心信息抽取出来，可以进行保存
9	解释器 Interpreter	定义语法解析规则
10	迭代器 Iterator	定义集合数据的遍历规则
11	访问者 Visitor	分离对象结构，与元素的执行算法

1. 观察者模式

Observer Design Pattern

1.1 引例：气象站信息发布系统-需求描述







1.1 引例：气象站信息发布系统-需求分析

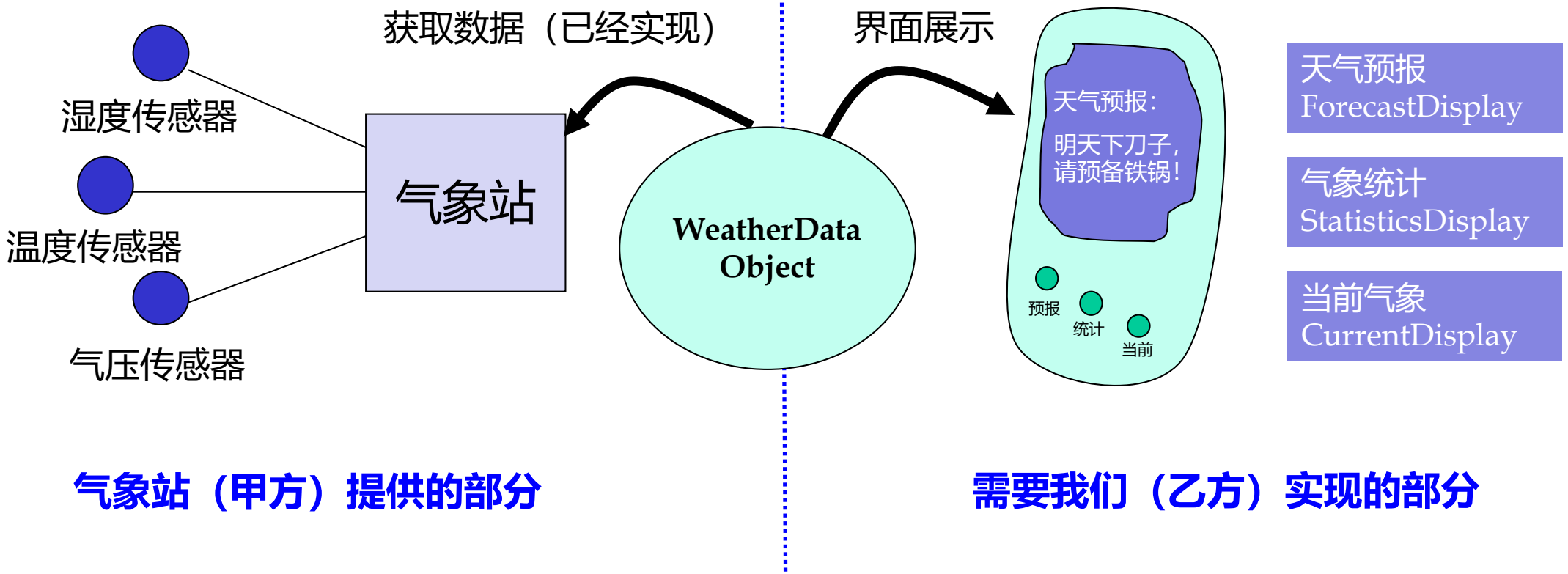
- 气象站提供了 WeatherData 对象，该对象能够获取目前的天气的测量值，包括：
  - 温度 Temperature
  - 湿度 Humidity
  - 气压 Pressure
- 即：气象站已经实现了如何获取温度、湿度和气压的方法。

1.1 引例：气象站信息发布系统-需求分析

- 需要乙方建立一个应用（Application），有三种显示界面：
  - 目前气象（如温度，湿度，气压值的显示）
  - 气象统计（如平均温度，平均湿度、最高温度的显示）
  - 简单预报（如短时间内的天气预报的显示）
- 对设计的要求
  - 三个界面必须在 WeatherData 对象 获取新的数据后，立刻更新
  - 该应用应该支持扩展，乙方应该提供API，以便其他的程序员写出自己的界面，并插入到该系统中

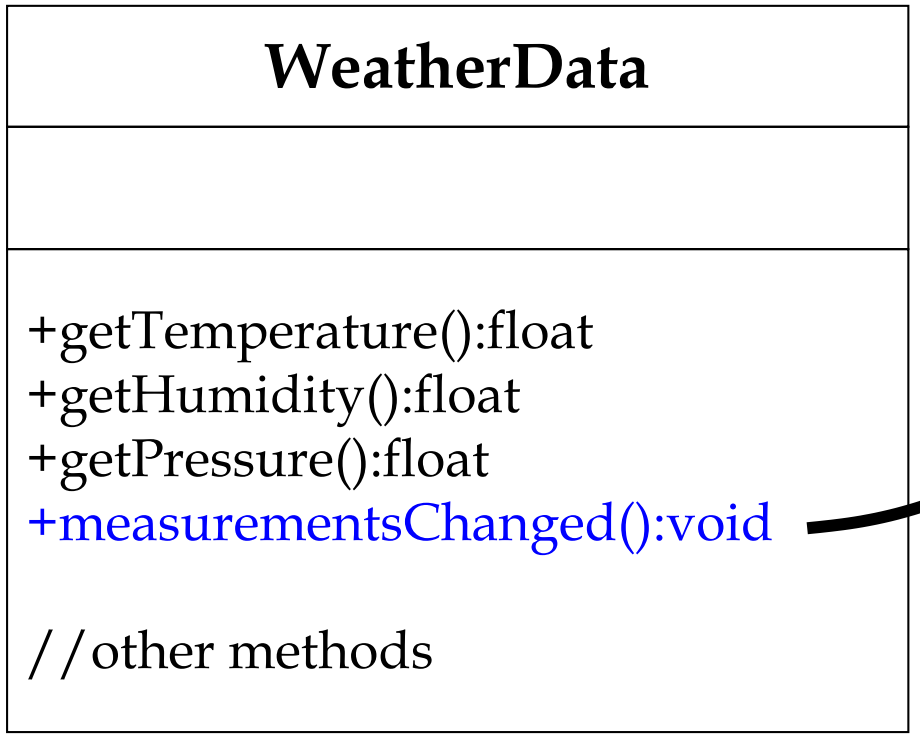
1.1 引例：气象站信息发布系统

- 需求分析



1.1 引例：气象站信息发布系统

- 甲方已提供的 WeatherData 类




```
/*
 * 当气象测量数据改变时, 调用该方法
*/

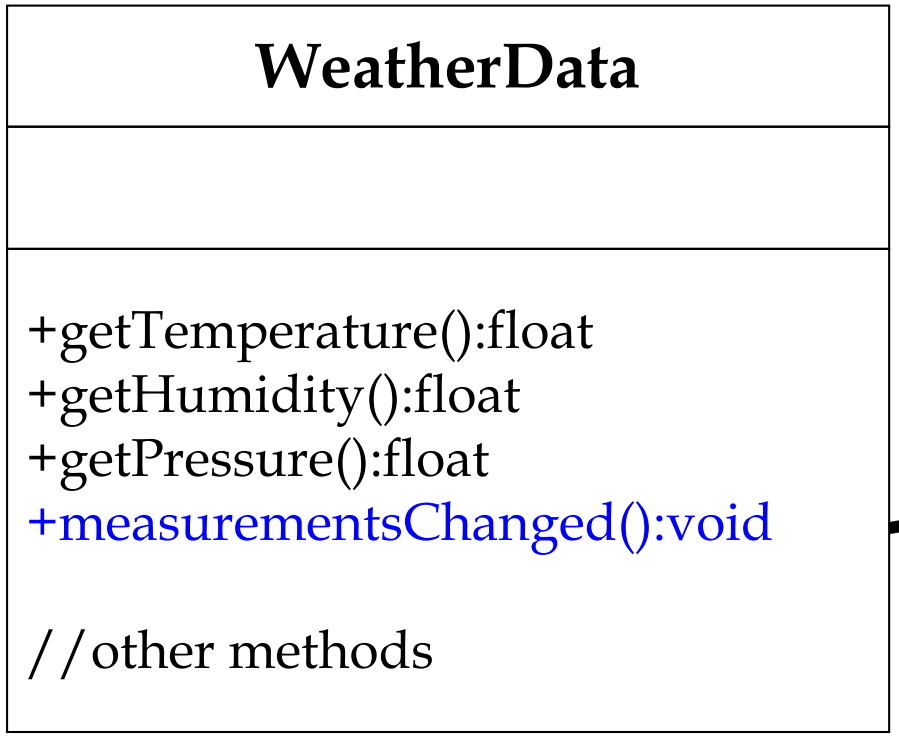
public void measurementsChanged() {

    //在此编写你的代码

}
```

1.1 引例：气象站信息发布系统

- 初步设计方案



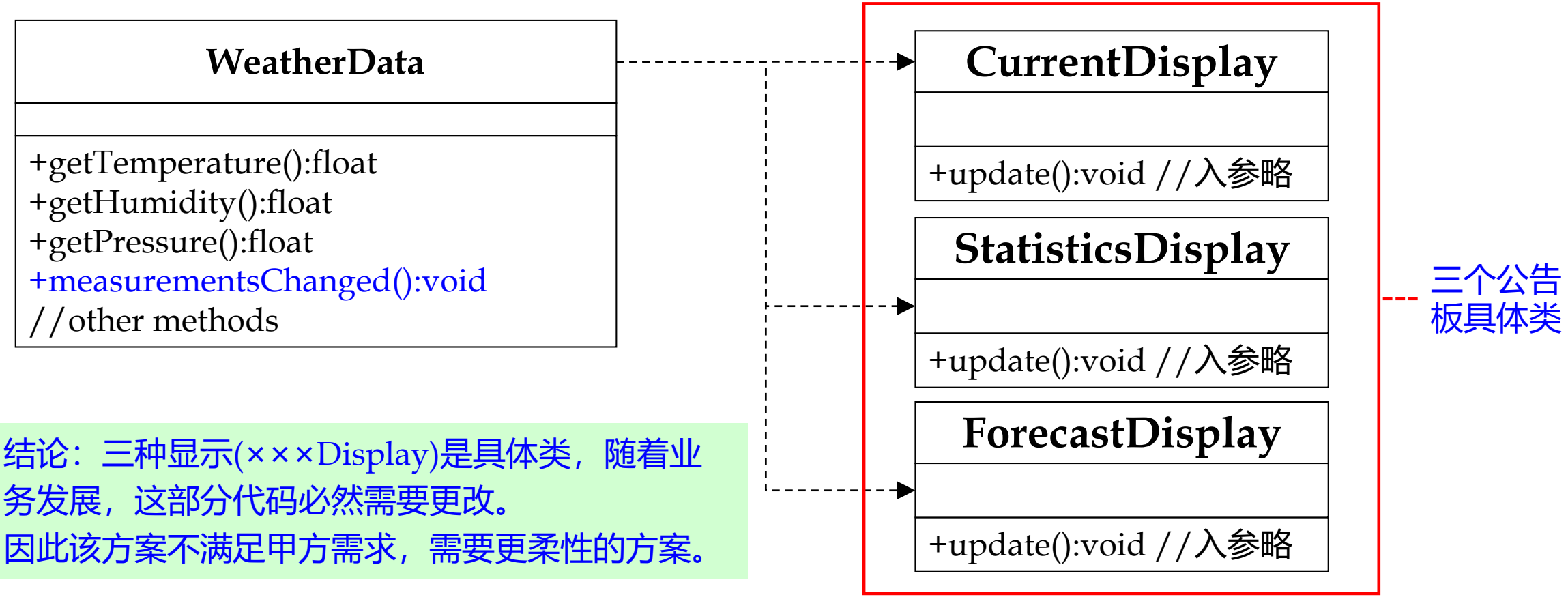

```
/*
 * 当气象测量数据改变时, 调用该方法
*/
public void measurementsChanged() {
    //在此编写你的代码

    //甲方提供的获取气象数据的方法如下：
    float temp = getTemperature();
    float humidity = getHumidity();
    float pressure=getPressure();

    //乙方初步的解决方案
    currentDisplay.update(temp, humidity, pressure);
    statisticsDisplay.update(temp, humidity, pressure);
    forecastDisplay.update(temp, humidity, pressure);
}
```

1.1 引例：气象站信息发布系统

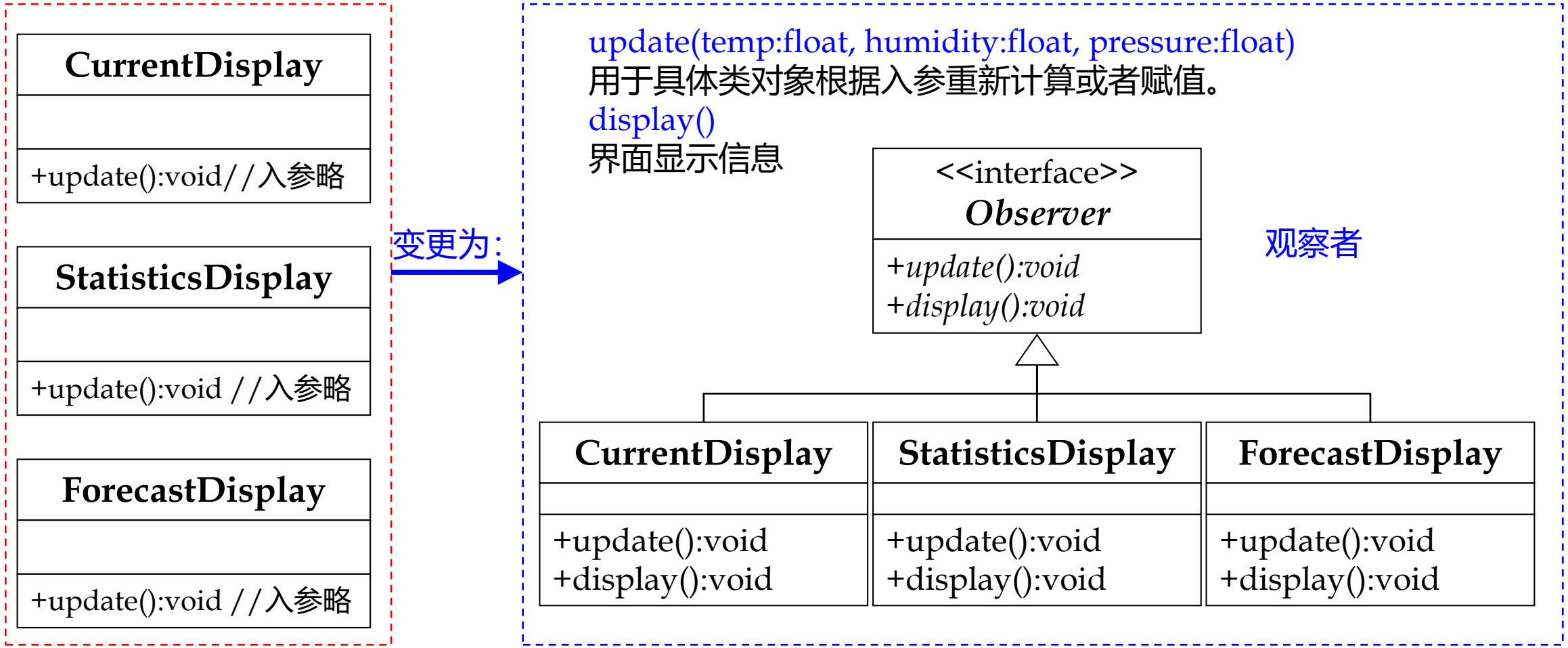
- 用类图方式表达类，及类间关联，讨论是否满足 OCP



结论：三种显示(×××Display)是具体类，随着业务发展，这部分代码必然需要更改。
因此该方案不满足甲方需求，需要更柔性的方案。

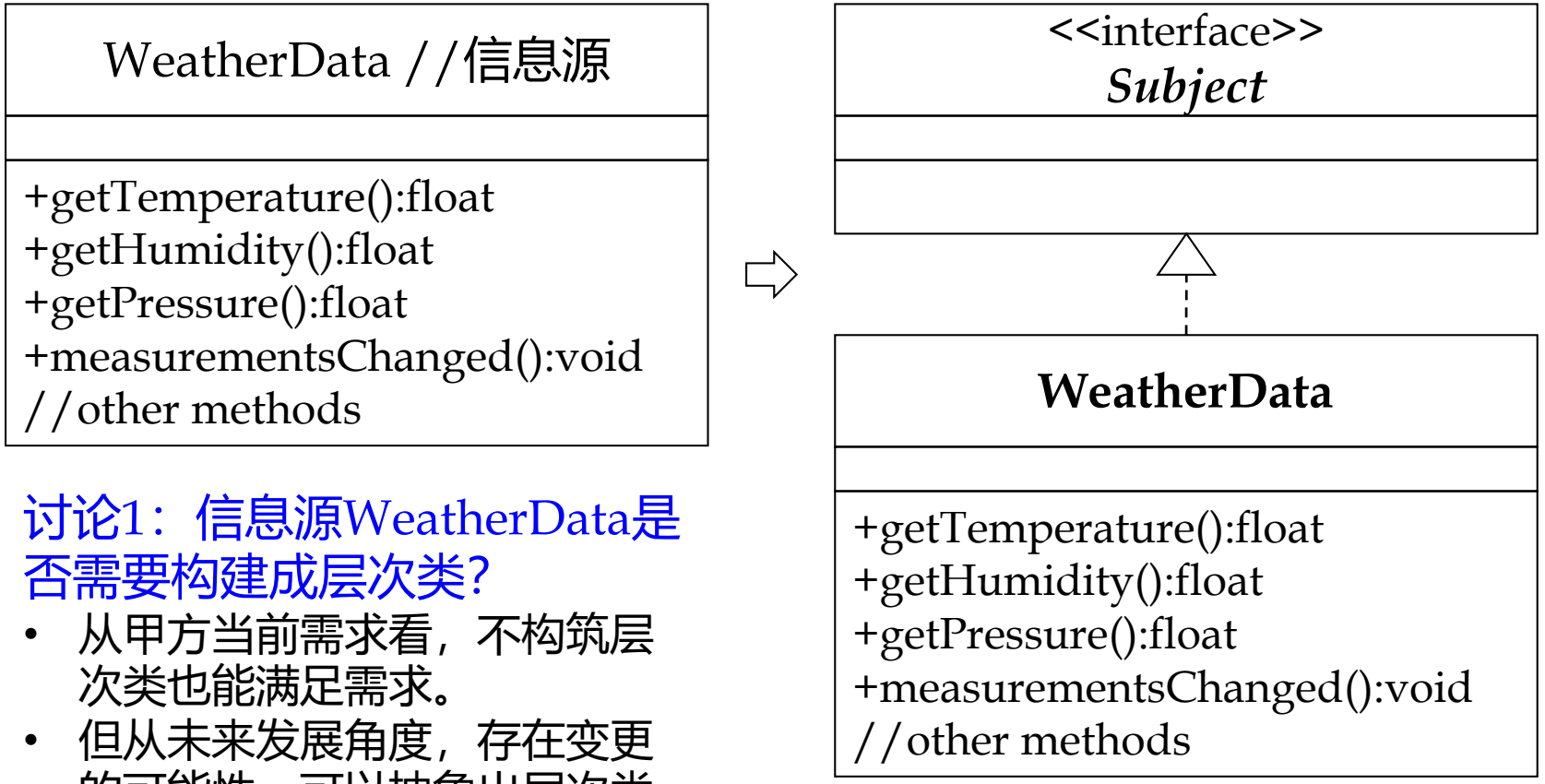
1.1 引例：气象站信息发布系统

- 重构-将公告板形成层次类



1.1 引例：气象站信息发布系统

- 重构：将信息源形成层次类



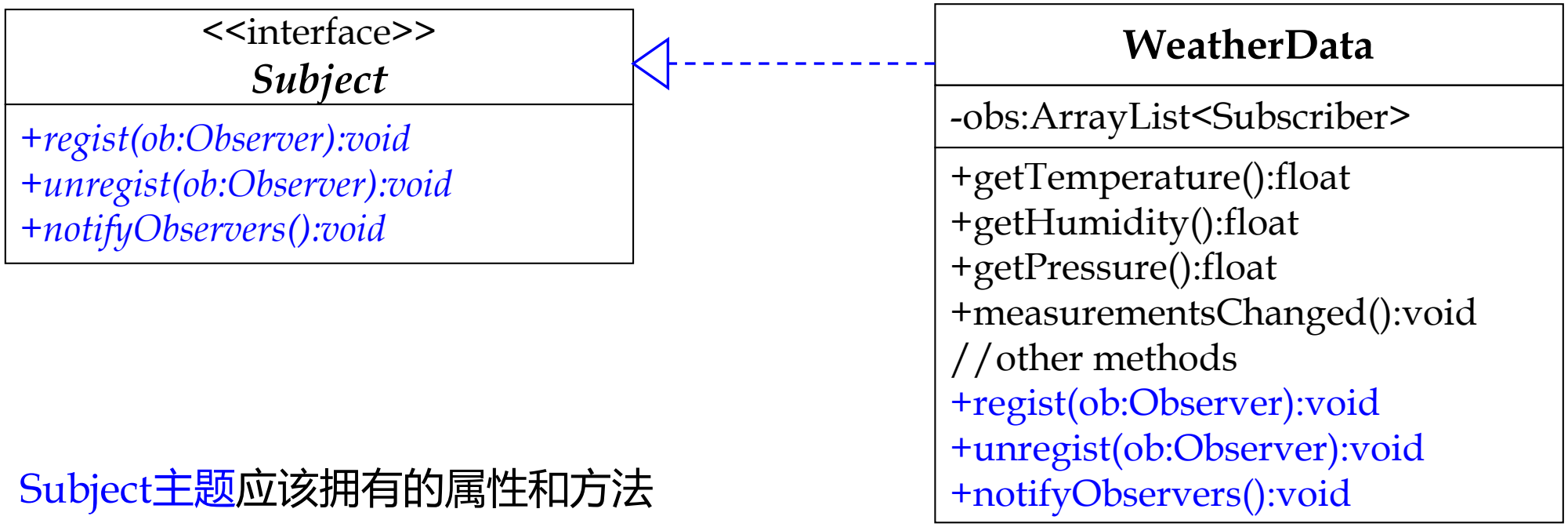
讨论1：信息源WeatherData是否需要构建成层次类?

- 从甲方当前需求看，不构筑层次类也能满足需求。
- 但从未来发展角度，存在变更的可能性，可以抽象出层次类

讨论2：Subject接口的职责有那些?

- 系统存在三种界面，而且界面都需要获得信息源的消息，因此需要一个界面的集合。
- 每个界面能够注册到这个集合，以便接收到新的消息
- 界面也可以从集合中取消，不再接收消息。
- 新的消息到来，能够通知到每一个界面

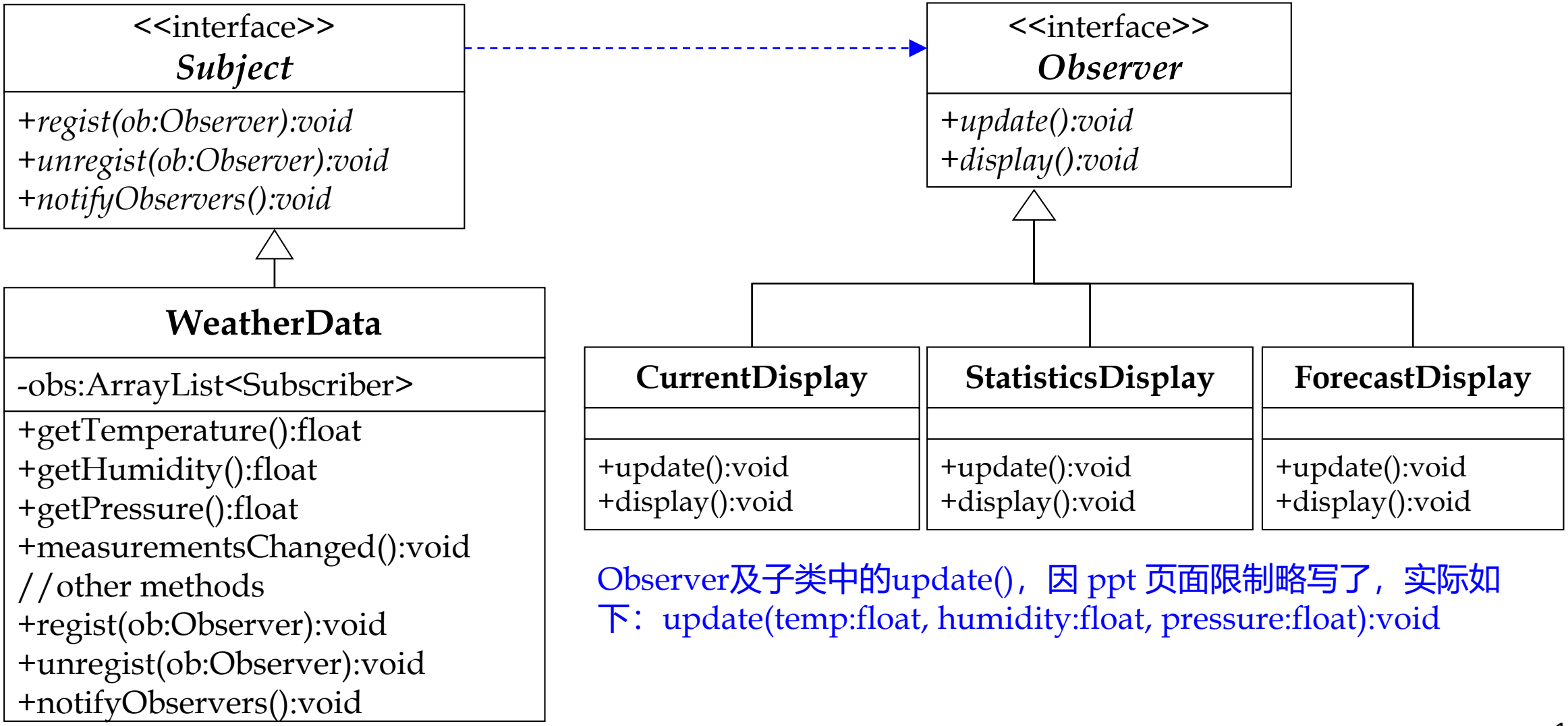
基于讨论1和讨论2，Subject接口的职责如下所示：



Subject主题应该拥有的属性和方法

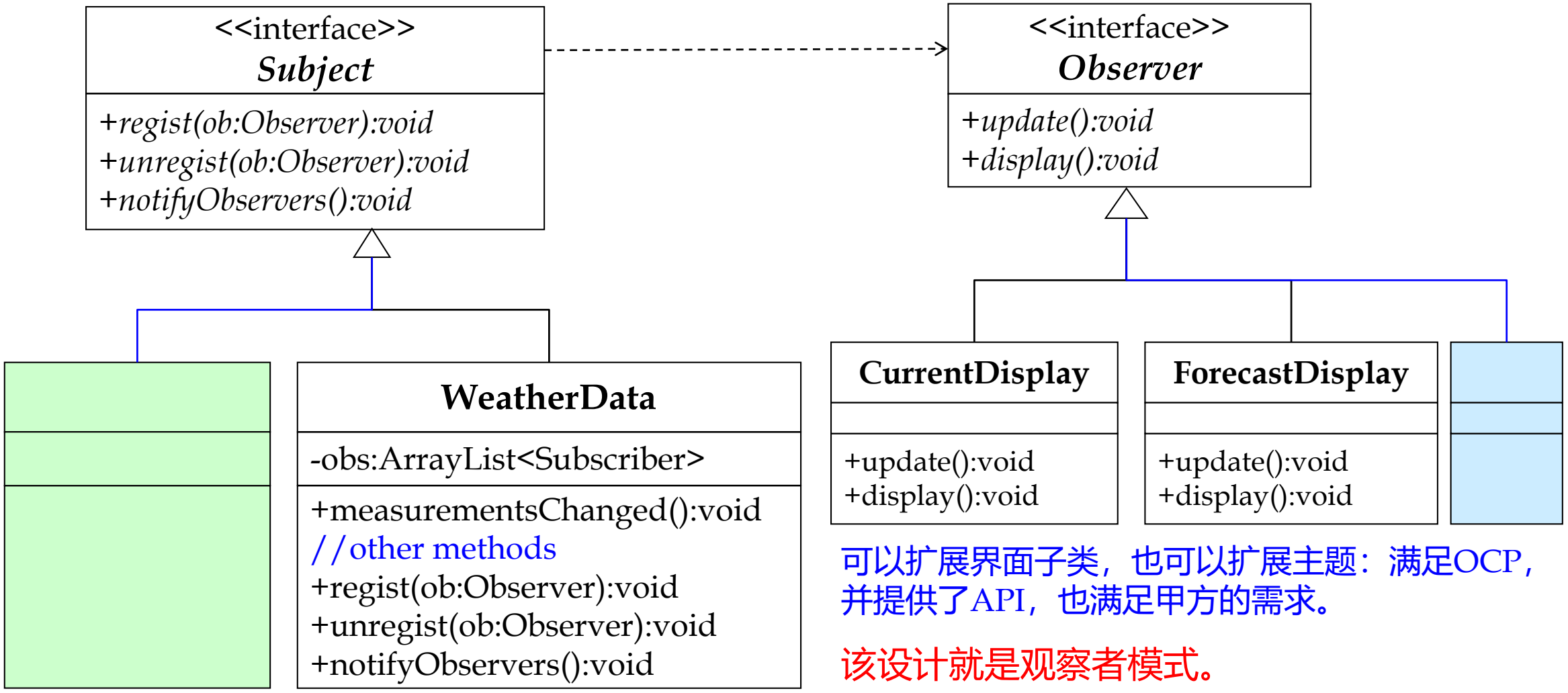
No.	属性或方法	说明
1	observers:ArrayList<Observers>	Subject需要知道都哪些界面定阅数据消息，即该对象需要持有需要消息的界面列表。但接口中无法定义属性，因此放到实现类中，对该集合的操作，也随之在实现类中
2	regist(ob:Observer):void	界面应该明确知道向谁订阅消息，因此需要注册
3	unregist(ob:Observer):void	允许界面取消订阅
4	notifyObservers():void	新的数据产生后，通知每一个订阅的界面

1.1 引例：气象站信息发布系统最终的设计方案



Observer及子类中的update()，因 ppt 页面限制略写了，实际如下：update(temp:float, humidity:float, pressure:float):void

1.1 引例：气象站信息发布系统最终的设计方案讨论



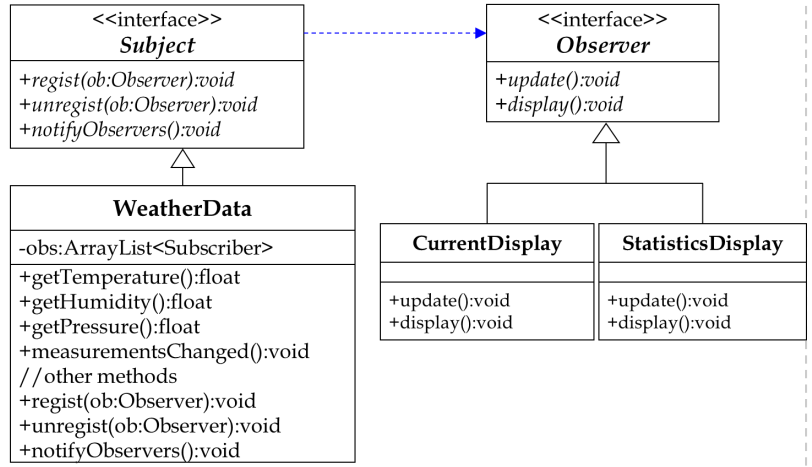
可以扩展界面子类，也可以扩展主题：满足OCP，并提供了API，也满足甲方的需求。

该设计就是观察者模式。

1.1 引例：气象站信息发布系统示意代码

```
public interface Observer {
    public void update(float temp, float humidity, float pressure);
    public void display();
}
```

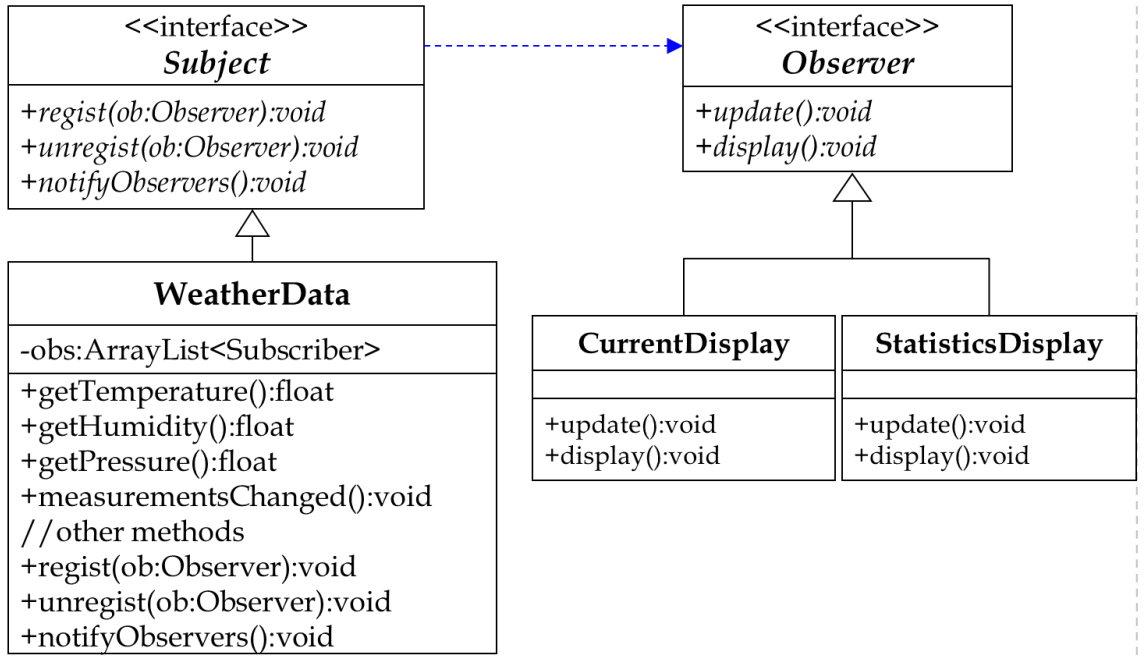
```
public class CurrentDisplay implements Observer {
    private float temperature;
    private float humidity;
    @Override
    public void update(float temp, float humidity, float pressure) {
        this.temperature = temp;
        this.humidity = humidity;
        display();
    }
    @Override
    public void display() {
        System.out.println("Current conditions: " + temperature
                + "F degrees and " + humidity + "% humidity");
    }
}
```



1.1 引例：气象站信息发布系统示意代码

```
public interface Subject {
    void regist(Observer o);
    void unRegist(Observer o);
    void notifyObservers();
}
```

```
public class WeatherData implements Subject {
    private ArrayList<Observer> observers = new ArrayList();
    private float temperature;
    private float humidity;
    private float pressure;
    @Override
    public void regist(Observer o) {
        observers.add(o);
    }
    @Override
    public void unRegist(Observer o) {
        int i = observers.indexOf(o);
        if (i >= 0) {
            observers.remove(i);
        }
    }
```



1.1 引例：气象站信息发布系统示意代码

```
    @Override
    public void notifyObservers() {
        for (int i = 0; i < observers.size(); i++) {
            Observer observer = observers.get(i);
            observer.update(temperature, humidity, pressure);
        }
    }

    public void measurementsChanged() {
        notifyObservers();
    }

    public void setMeasurements(float temperature, float humidity, float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        this.pressure = pressure;
        measurementsChanged();
    }
    // other WeatherData methods here
}
```

1.1 引例：气象站信息发布系统示意代码

```
public class WeatherStation {
    public static void main(String[] args) {
        WeatherData weatherData = new WeatherData();

        // 创建公告板对象
        CurrentDisplay currentDisplay = new CurrentDisplay();
        StatisticsDisplay statisticsDisplay = new StatisticsDisplay();
        ForecastDisplay forecastDisplay = new ForecastDisplay();

        // 订阅气象站消息
        weatherData.regist(currentDisplay);
        weatherData.regist(statisticsDisplay);
        weatherData.regist(forecastDisplay);

        // 气象站发布新消息
        weatherData.setMeasurements(80, 65, 30.4f);
        weatherData.setMeasurements(82, 70, 29.2f);
        weatherData.setMeasurements(78, 90, 29.2f);
    }
}
```

1.1 引例：气象站信息发布系统示意代码

- 运行结果

```
Current conditions: 80.0F degrees and 65.0% humidity
Avg/Max/Min temperature = 80.0/80.0/80.0
Forecast: Improving weather on the way!

Current conditions: 82.0F degrees and 70.0% humidity
Avg/Max/Min temperature = 81.0/82.0/80.0
Forecast: Watch out for cooler, rainy weather

Current conditions: 78.0F degrees and 90.0% humidity
Avg/Max/Min temperature = 80.0/82.0/78.0
Forecast: More of the same
```

1.2 观察者模式原理

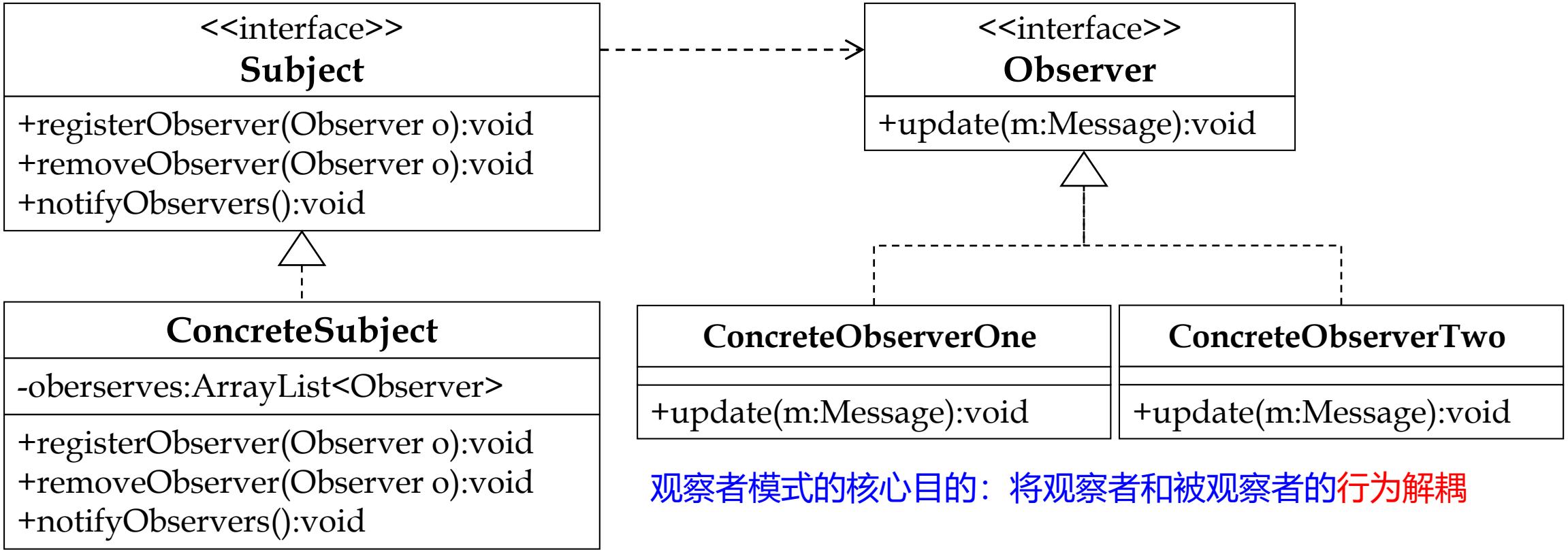
- 观察者模式（Observer Design Pattern）也被称为发布订阅模式（Publish-Subscribe Design Pattern）。在 GoF 的《设计模式》一书中，它的定义是这样的：

> Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.
> 在对象之间定义一个一对多的依赖，当一个对象状态改变的时候，所有依赖的对象都会自动收到通知

- 被依赖的对象叫作被观察者（Observable），依赖的对象叫作观察者（Observer）。
  - 实际项目开发中，这两种对象的称呼比较灵活：Subject-Observer、Publisher-Subscriber、Producer-Consumer、EventEmitter-EventListener、Dispatcher-Listener
  - 不管怎么称呼，只要应用场景符合刚刚给出的定义，都可以看作观察者模式。

Subject：抽象主题（抽象被观察者），抽象主题角色把所有观察者对象保存在一个集合里，每个主题都可以有任意数量的观察者，抽象主题提供一个接口，可以增加和删除观察者对象

Observer：抽象观察者，是观察者的抽象类，它定义了一个更新接口，使得在得到主题更改通知时更新自己。



观察者模式的核心目的：将观察者和被观察者的行为解耦

ConcreteSubject：具体主题（具体被观察者），该角色将有关状态存入具体观察者对象，在具体主题的内部状态发生改变时，给所有注册过的观察者发送通知。

ConcrereObserver：具体观察者，实现抽象观察者定义的更新接口，以便在得到主题更改通知时更新自身的状态。

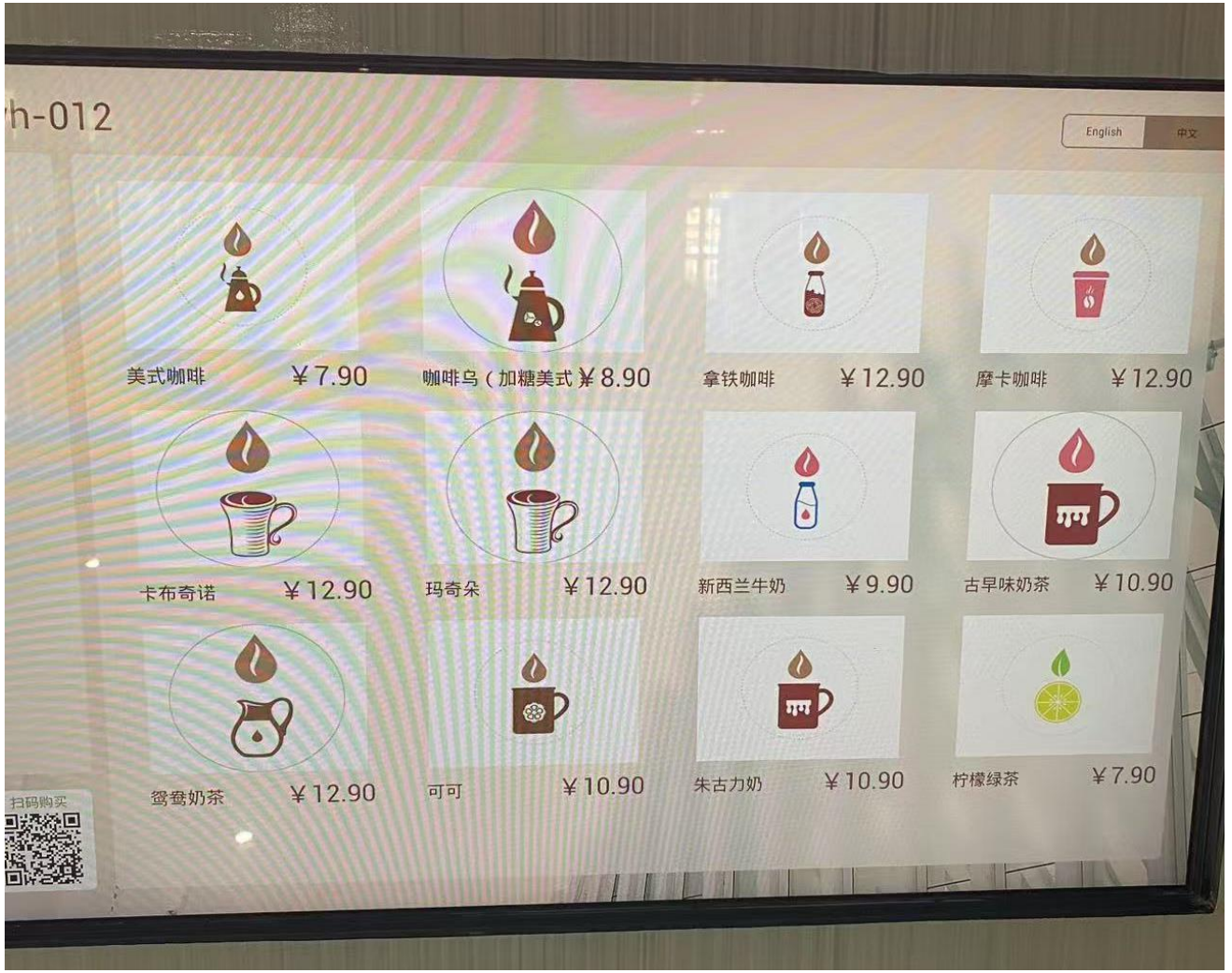
1.3 观察者模式基于不同应用场景的不同实现方式

- 观察者模式的应用场景非常广泛，小到代码层面的解耦，大到架构层面的系统解耦，再或者一些产品的设计思路，都有这种模式的影子，比如：
  - 邮件订阅（eg： github 的邮件订阅）
  - 各种网站的的动态、关注等（哔哩哔哩）
  - 微信关注（朋友圈、视频号等）
  - 消息队列（Message Queue，比如 ActiveMQ，Kafka，RocketMQ）
  - 物联网中的 MQTT 协议
- 本质上都是观察者模式。

2. 模板方法设计模式
Template Method Design Pattern

2.1 引例：热饮贩卖机自动冲泡组件设计与实现





2025.08 拍摄于研究院1号楼大厅北侧。在M楼一楼也有类似的贩卖机。

2.1 引例：热饮贩卖机自动冲泡组件设计与实现

- 某公司计划推出一款热饮贩卖机。该售卖机能够售卖两种饮料：热咖啡、热茶。两种饮料的冲泡方法如下：

咖啡冲泡法

(1) 把水煮沸（boil water）
(2) 用沸水冲泡咖啡（brew coffee grinds）
(3) 把咖啡倒进杯子（pour in cup）
(4) 加糖和牛奶（add sugar and milk）

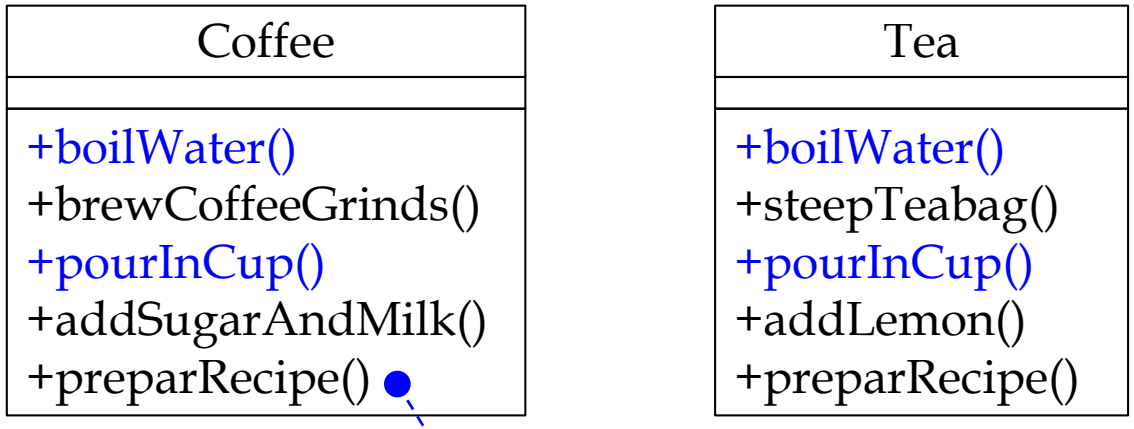
茶冲泡法

(1) 把水煮沸（boil water）
(2) 用沸水冲泡茶叶（steep tea bag）
(3) 把茶倒进杯子（pour in cup）
(4) 加柠檬（add lemon）

- 根据以上冲泡流程，为热饮贩卖机编写代码，实现饮料的自动泡制

2.1 引例：热饮贩卖机自动冲泡组件设计与实现

- 很自然的，能够想到将 coffee 和 tea 设计成具体类，再进行抽象

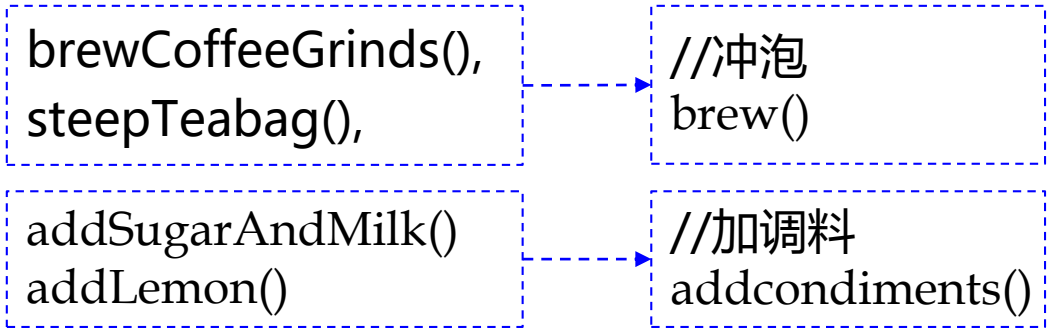



```
prepareRecipe() {
    boilWater(); //Step1烧水
    brewCoffeeGrinds(); //Step2冲泡
    pourInCup(); //Step3倒杯子中
    addSugarAndMilk(); //Step4加调料
}
```

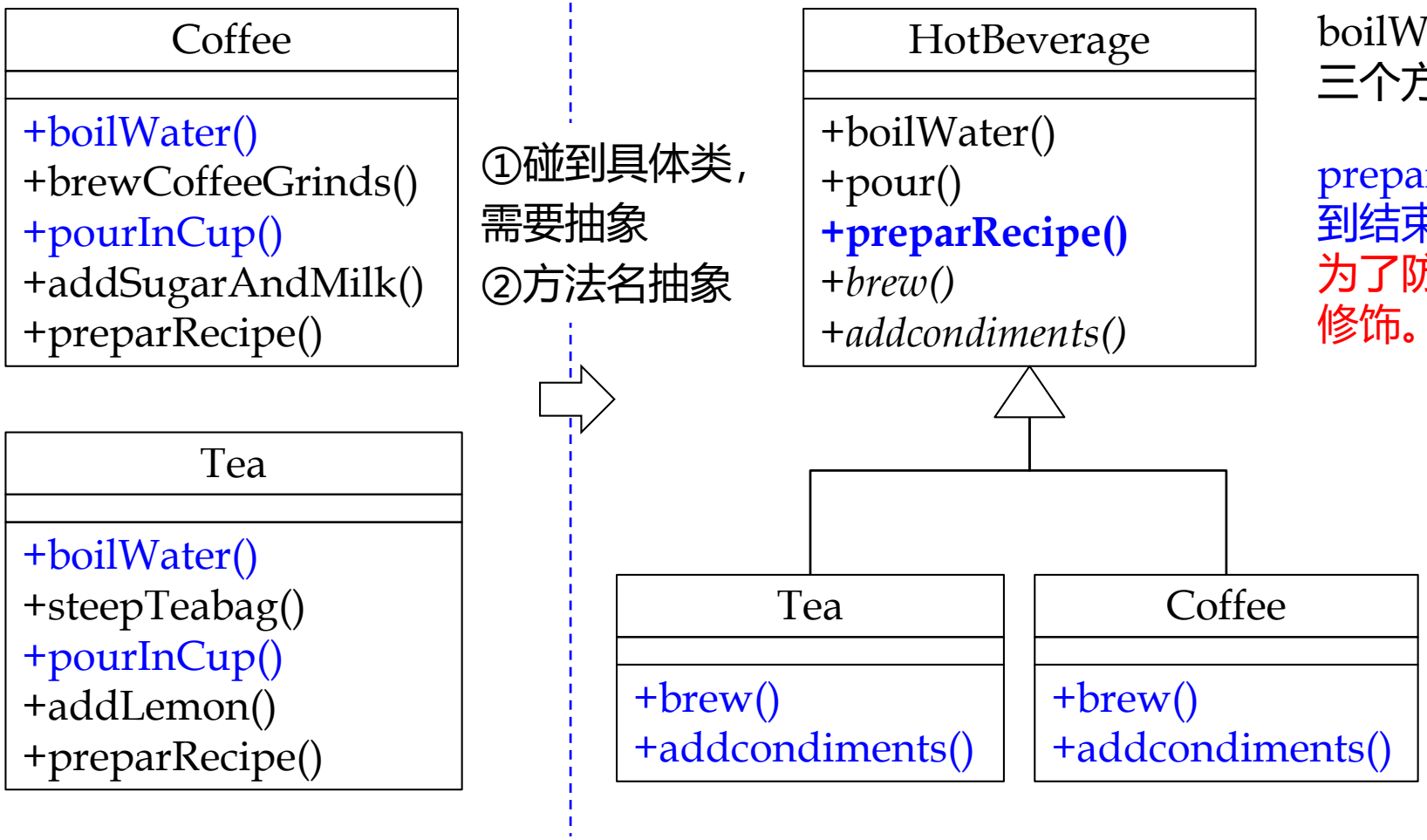
prepareRecipe() 控制其他方法的执行顺序

定义类中方法时，也要有抽象思维，不要暴露实现细节。即方法的命名应该具有一定的抽象性

brewCoffeeGrinds(), addSugarAndMilk()
steepTeabag(), addLemon() 这样的过于具体，需要抽象



2.1 引例：热饮贩卖机自动冲泡组件设计与实现



boilWater()，pour()，preparRecipe()三个方法代码可以继承复用

preparRecipe() 控制住了热饮从开始到结束的流程，被称为模板方法

为了防止被子类篡改，可以加 final 修饰。

brew()和addcondiments()推迟到子类实现。

2.1 引例：热饮贩卖机自动冲泡组件设计与实现

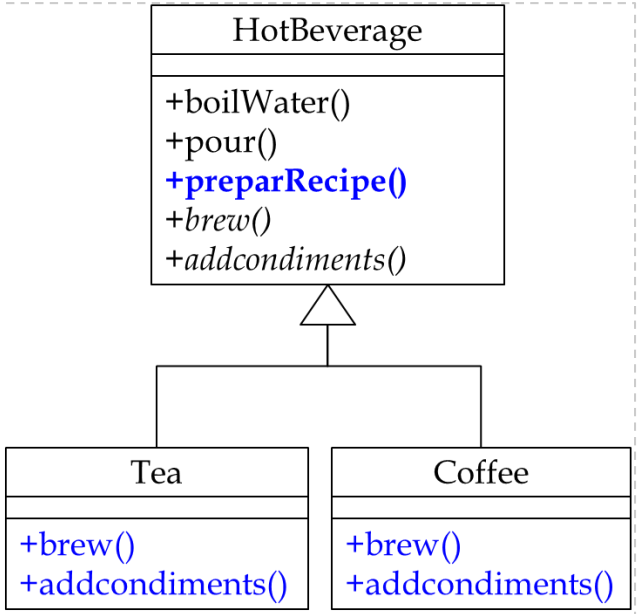
```
public abstract class HotBeverage {
    public void boilWater() {
        System.out.println("Boiling water!");
    }

    public void pourIncup() {
        System.out.println("Pouring into water!");
    }

    public abstract void brew();

    public abstract void addcondiments();

    public void preparRecipe() {
        boilWater();
        brew();
        pourIncup();
        addcondiments();
    }
}
```

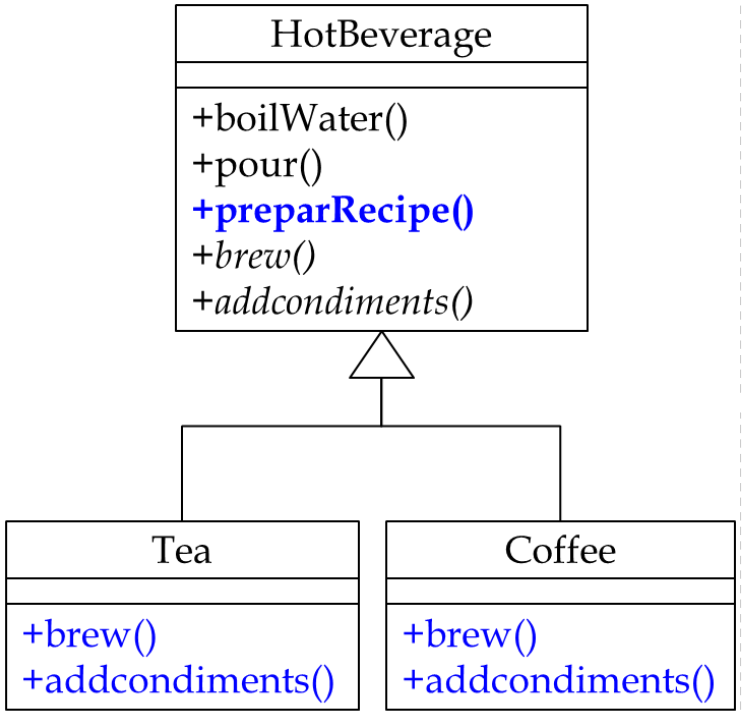


模板方法-将执行流程控制住。

2.1 引例：热饮贩卖机自动冲泡组件设计与实现

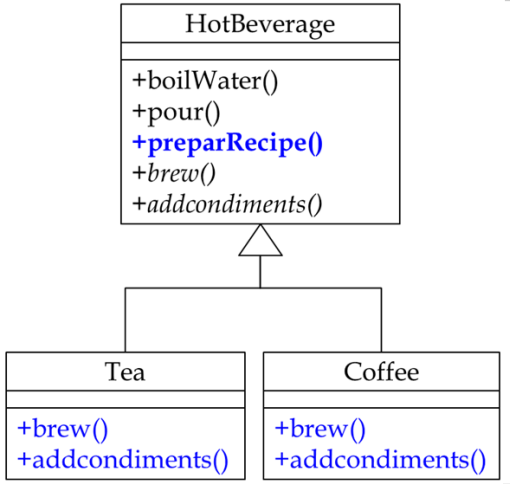
```java
public class Coffee extends HotBeverage{
    @Override
    public void brew() {
        System.out.println("Dripping Coffee through filter!");
    }
    @Override
    public void addcondiments() {
        System.out.println("Adding Sugar and Milk");
    }
}
```

```java
public class Tea extends HotBeverage {
    @Override
    public void brew() {
        System.out.println("Steeping the tea!");
    }
    @Override
    public void addcondiments() {
        System.out.println("Adding Lemon!");
    }
}
```



2.1 引例：热饮贩卖机自动冲泡组件设计与实现

```java
public class Client {
    public static void main(String[] args) {
        HotBeverage coffee = new Coffee();
        coffee.preparRecipe();
        System.out.println("==============");
        HotBeverage tea = new Tea();
        tea.preparRecipe();
    }
}
```




```
D:\tools\jdk-17.0.9\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.1.1.1\lib\idea_rt.jar=4227" -Df
Boiling water!
Dripping Coffee through filter!
Pouring into water!
Adding Sugar and Milk
==============
Boiling water!
Steeping the tea!
Pouring into water!
Adding Lemon!
```

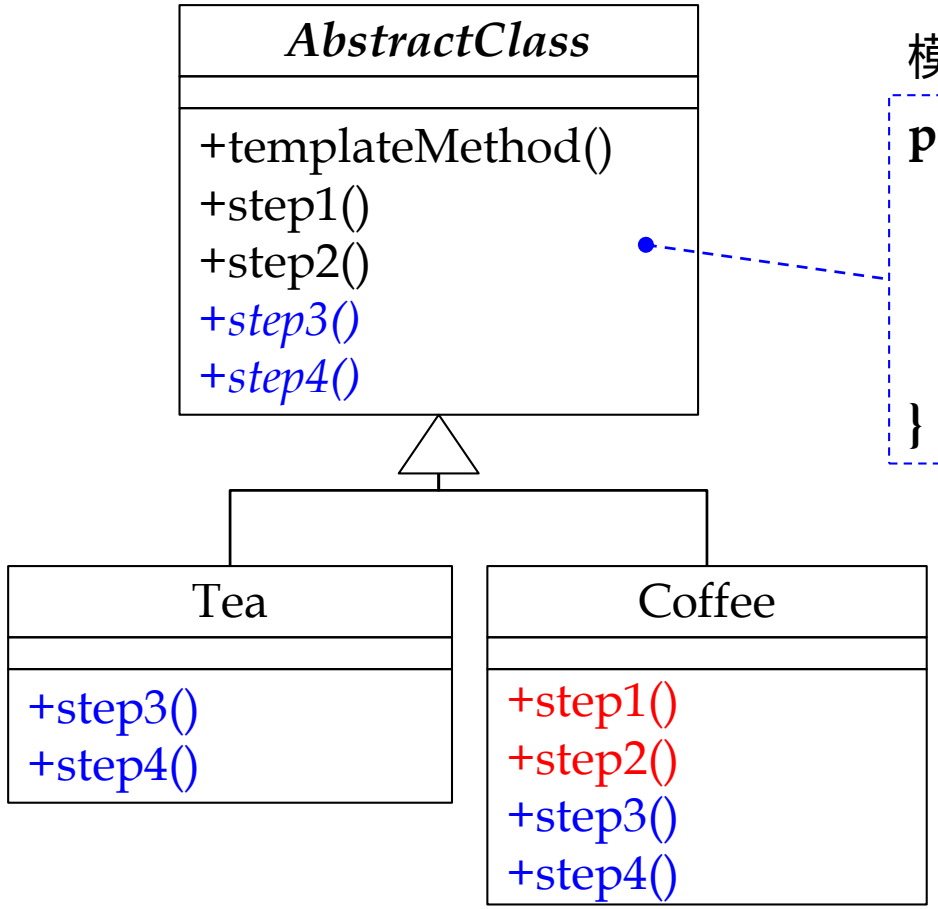
2.2 模板方法设计模式-定义

- 模板方法设计模式，英文是 Template Method Design Pattern。在GoF 《设计模式》一书中，定义如下：

> Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.
> 模板方法模式在一个方法中定义一个算法骨架，并将某些步骤推迟到子类中实现。模板方法模式可以让子类在不改变算法整体结构的情况下，重新定义算法中的某些步骤。

- 这里的“算法”，可以理解为广义上的“业务逻辑”，并不特指数据结构和算法中的“算法”
- 这里的算法骨架就是“模板”，包含算法骨架的方法就是“模板方法”，这也是模板方法模式名字的由来。

2.2 模板方法设计模式-典型结构



模板方法定义了算法骨架

```
public void templateMethod(){
    step1;
    step2;
    step3;
    step4;
}
```

并将某些步骤推迟到子类中实现，例如：step3，step4

子类在不改变算法整体结构的情况下，重新定义算法中的某些步骤。例如 step1，step2

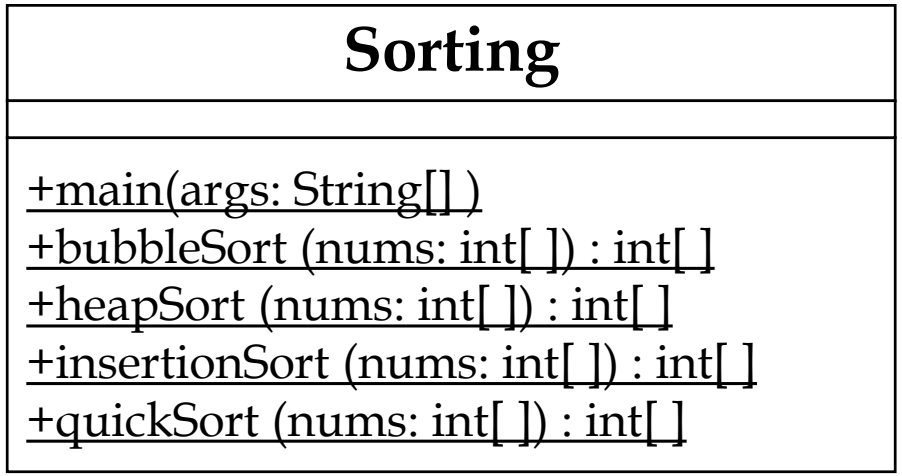
3. 策略模式
Strategy Design Pattern

3.1 引例：排序程序的设计与实现

- 假设我们要设计一个排序程序，能够使用如下几种排序算法：
  - 冒泡排序算法 Bubble Sort
  - 堆排序算法 Heap Sort
  - 插入排序算法 Insertion Sort
  - 快速排序算法 Quick Sort

3.1 引例：排序程序的设计与实现

- 设计方案1：使用单独的类实现所有算法



备注：main方法需要调用其他方法，因此其他方法需要为 static，采用下划线表示 static

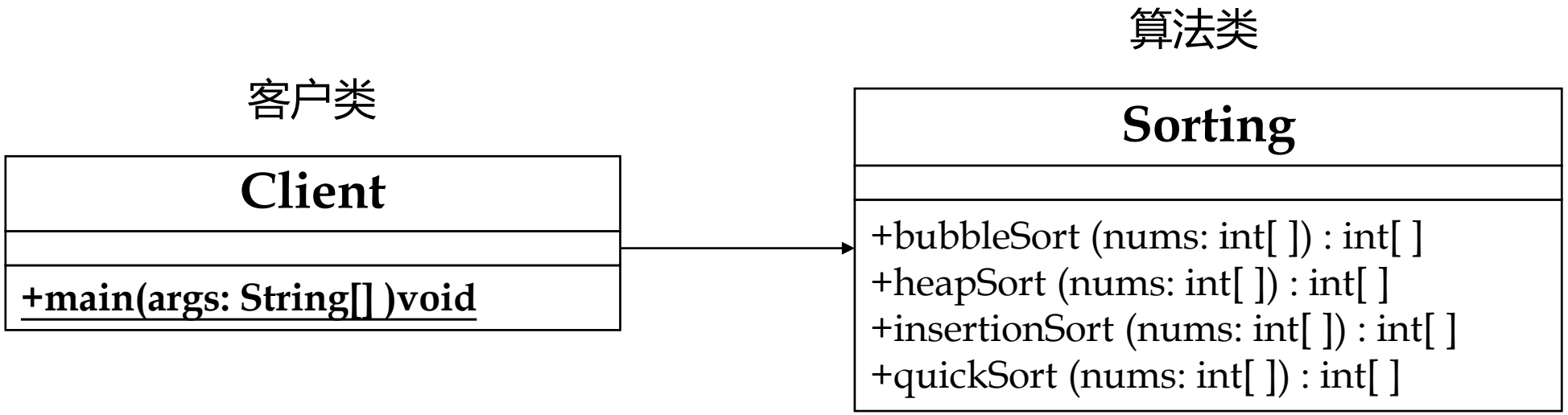
这是最糟糕的设计方案

✓ 如果要增加一个新算法，或修改某个已有算法则需要重新编译整个类.

✓ 可复用性差，Sorting 类不可以被直接复用

3.1 引例：排序程序的设计与实现

- 设计方案2：分离出客户类和算法类



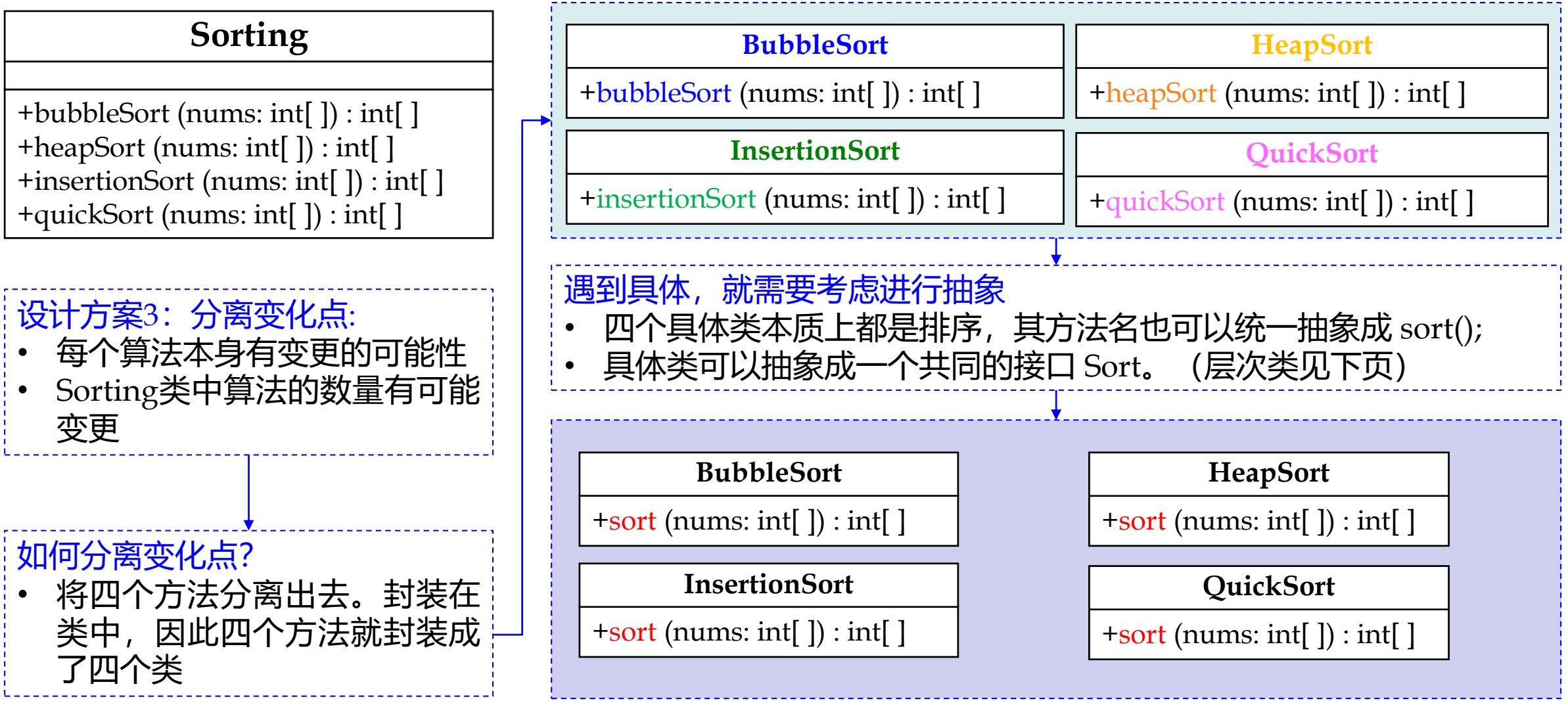
优点：Sorting类可以被复用

缺点：不满足开闭原则

扩展：若要增加一个新算法，则需要重新编译 Sorting 类；

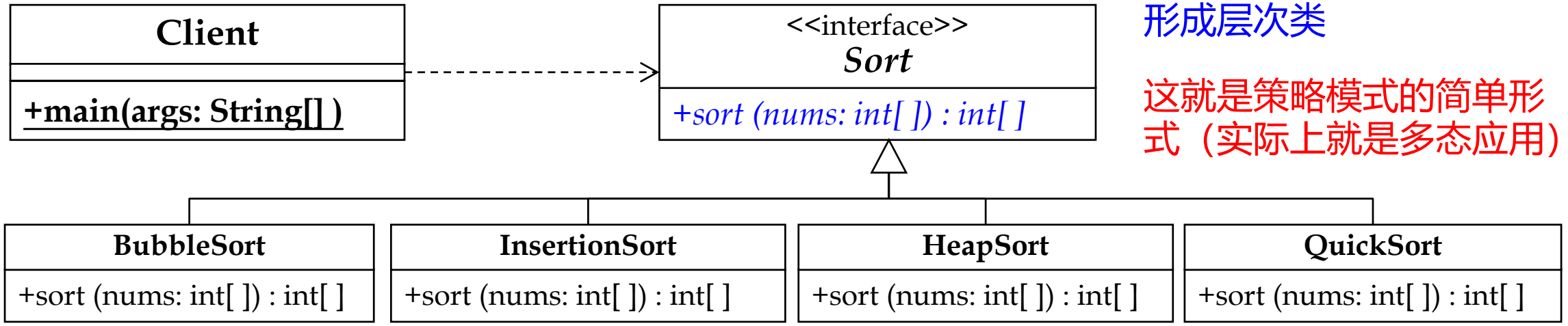
修改：若要修改某个算法代码，则需要重新编译 Sorting 类

3.1 引例：排序程序的设计与实现



3.1 引例：排序程序的设计与实现

- 设计方案3：分离变化点



形成层次类

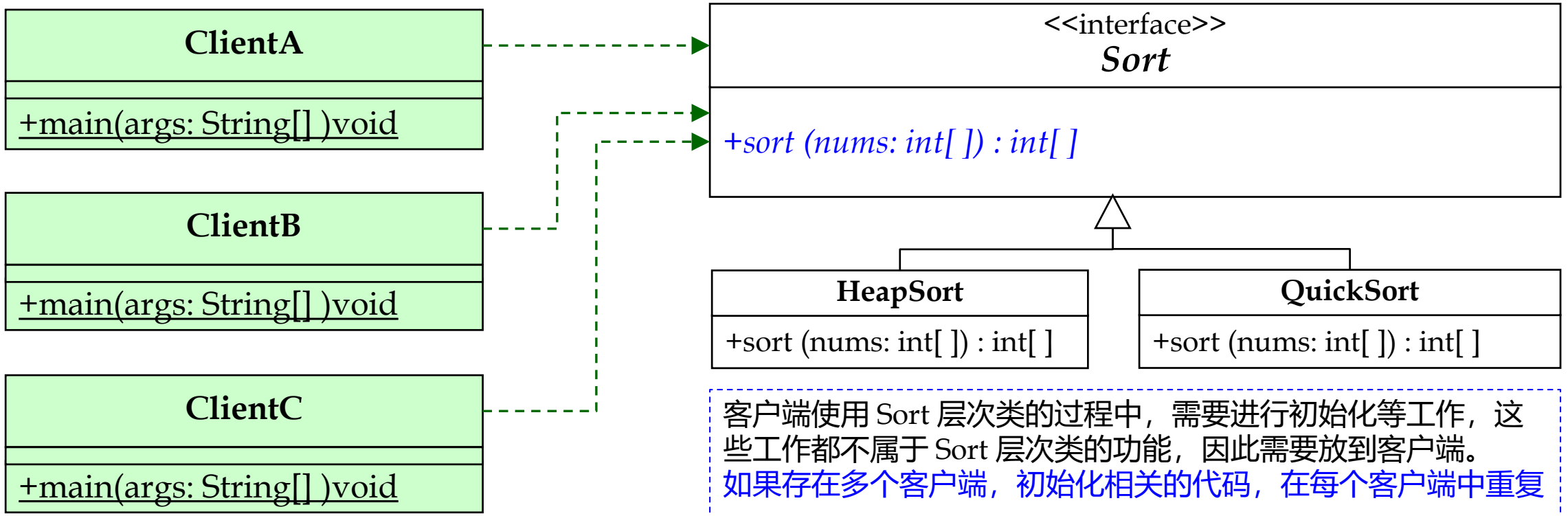
这就是策略模式的简单形式（实际上就是多态应用）

设计优点-满足开闭原则、依赖倒置原则：

- 当添加新算法或修改某个已有算法时，不需要更改和重新编译现有的代码
- Client 类和算法实现类都依赖抽象接口程序 Sort，而不是依赖于具体
- 要使用某个算法，在 Client 中的 Sort 对象实例化某个具体的子类即可。

3.1 引例：排序程序的设计与实现

- 设计方案3：分离变化点存在的问题：

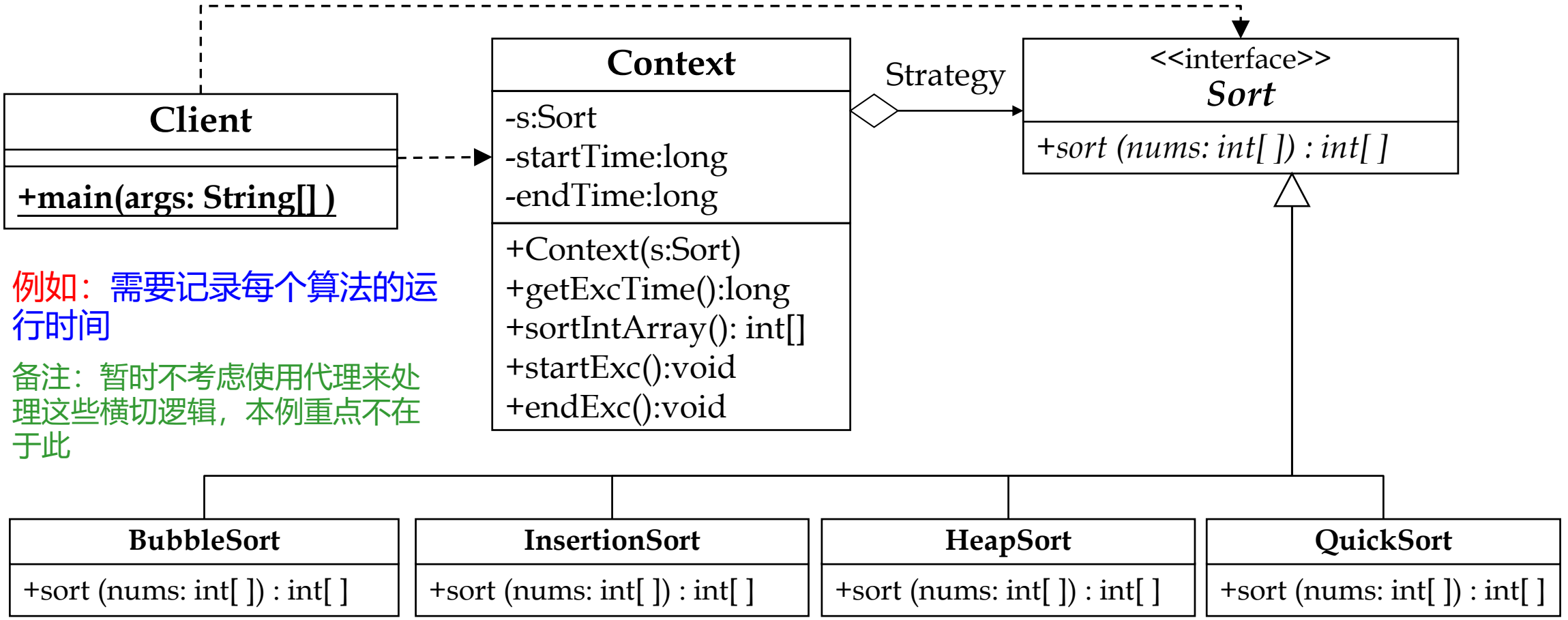


客户端使用 Sort 层次类的过程中，需要进行初始化等工作，这些工作都不属于 Sort 层次类的功能，因此需要放到客户端。
如果存在多个客户端，初始化相关的代码，在每个客户端中重复

因此可以在层次类和客户端之间，加上一个负责初始化，或者全局控制的类，负责协调 Client 和 Sort 层次类，这个类一般称为环境类，Context

3.1 引例：排序程序的设计与实现

- 设计方案4：增加环境类



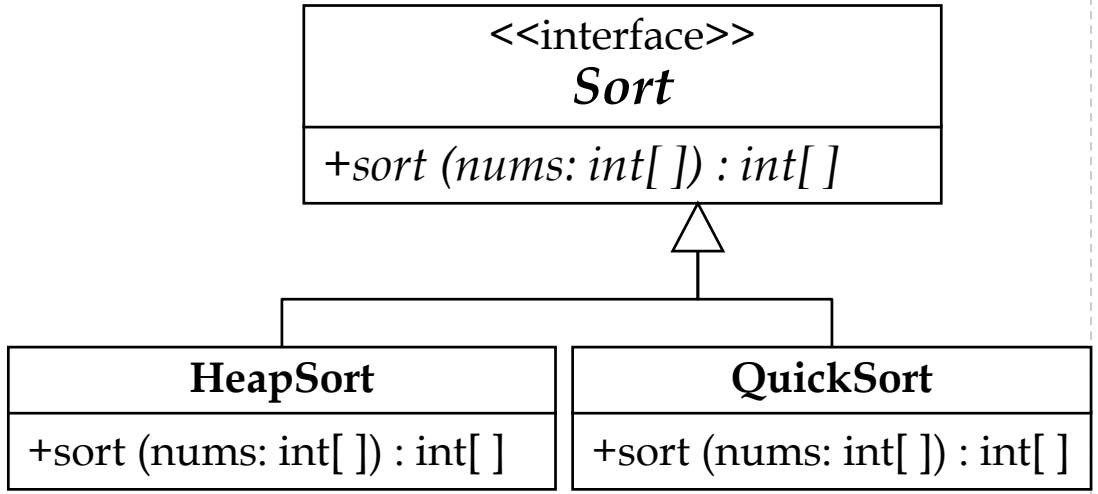
例如：需要记录每个算法的运行时间

备注：暂时不考虑使用代理来处理这些横切逻辑，本例重点不在于此

3.1 引例：排序程序的设计与实现

```
public interface Sort {
    int[] sort(int[] num);
}
```

```
public class BubbleSort implements Sort{ //其他排序类代码自行实现
    @Override
    public int[] sort(int[] num) {
        int n = num.length;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (num[j] > num[j + 1]) {
                    int temp = num[j];
                    num[j] = num[j + 1];
                    num[j + 1] = temp;
                }
            }
        }
        return num;
    }
}
```



3.1 引例：排序程序的设计与实现

```
public class Context {
    private Sort s;
    private long startTime;
    private long endTime;
    public Context(Sort s) {
        this.s = s;
    }
    public void startExc() {
        startTime = System.currentTimeMillis();
    }
    public void endExc() {
        endTime = System.currentTimeMillis();
    }
    public long getExcTime() {
        long exeTime = 0;
        exeTime = endTime - startTime;
        return exeTime;
    }
    public int[] sortIntArray(int[] a) {
        return s.sort(a);
    }
}
```

```
public class Client {
    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};
        Sort s = new BubbleSort();
        Context con = new Context(s);
        con.startExc();
        arr = con.sortIntArray(arr);
        con.endExc();
        System.out.println(Arrays.toString(arr));
        System.out.println(con.getExcTime());
    }
}
```

- 由于策略类都实现同一个接口，所以使它们之间可以自由切换；
- 添加一个具体的策略类即可，不需要改变原有的代码，符合“开闭原则“
- 环境类持有策略类的对象。具体的策略对象由客户端设置

```java
public class Client {
    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};
        Sort s = new BubbleSort();
        Context con = new Context(s);
        con.startExc();
        arr = con.sortIntArray(arr);
        con.endExc();
        System.out.println(Arrays.toString(arr));
        System.out.println(con.getExcTime());
    }
}
```

- 继续观察以上代码：

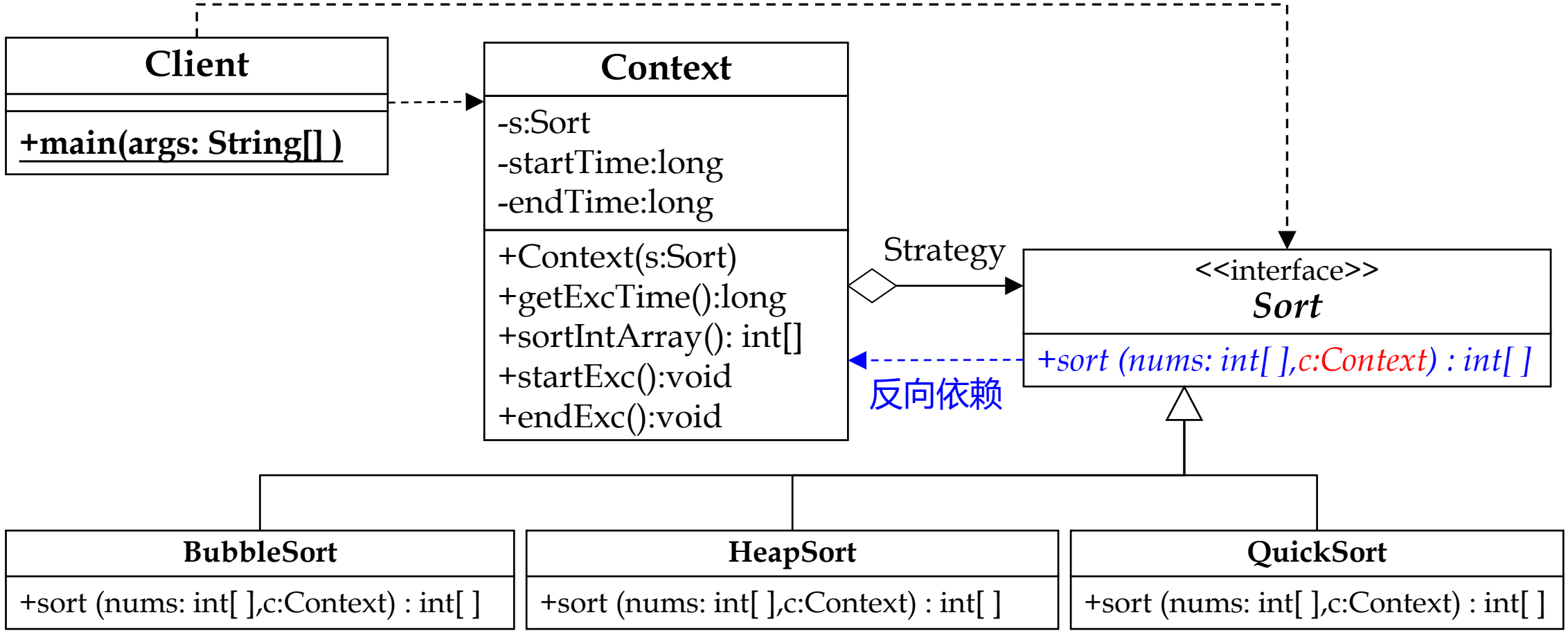
con.startExc() 和 con.endExc() 应该在排序代码前后调用。是否可以放到 sort 中处理?

```java
public int[] sort(int[] intArray, Context ct){
    ct.startExc();
    // 省略真实的排序代码（业务逻辑）
    ct.endExc();
    return intArray;
}
```

此时，sort() 方法中需要传入 Context 对象，否则没有办法使用 Context 中的 startExc() 和 endExc() 方法

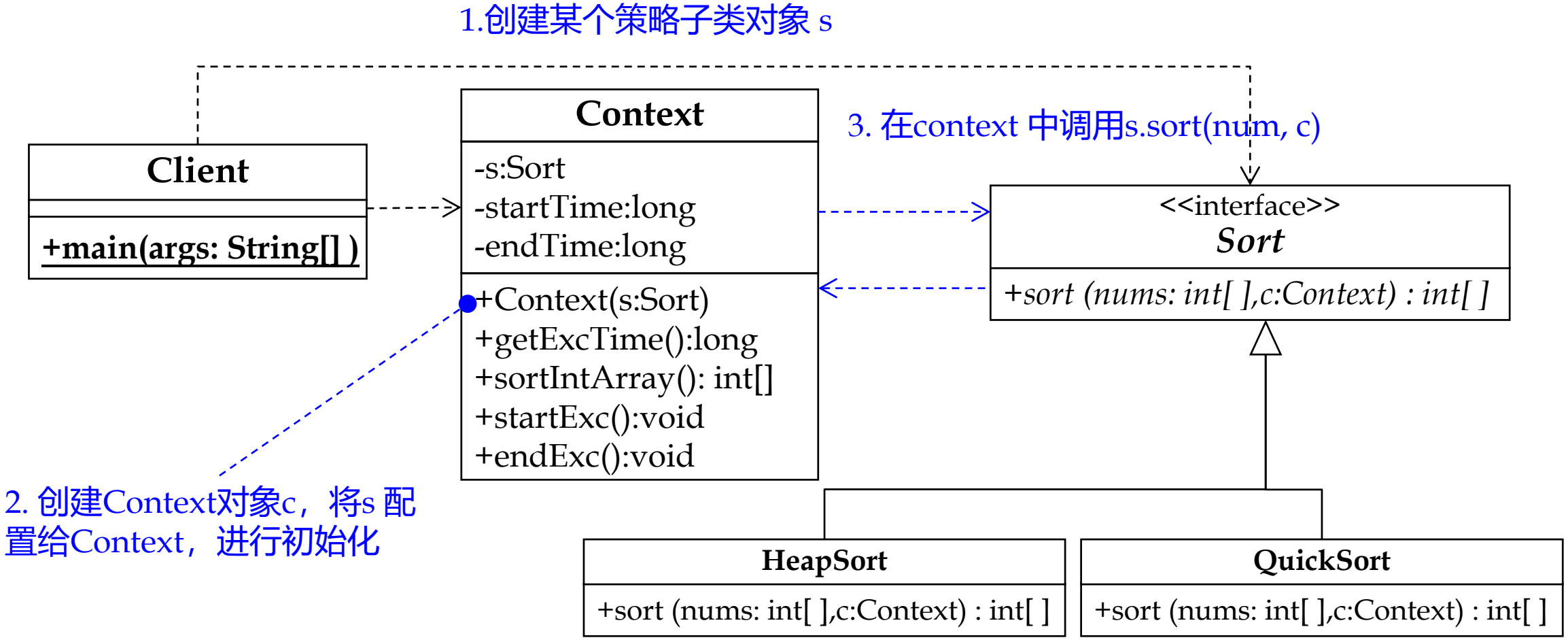
3.1 引例：排序程序的设计与实现

- 设计方案5-灵活应用策略模式，增加反向依赖



3.1 引例：排序程序的设计与实现

- 设计方案5-灵活应用策略模式，增加反向依赖



3.1 引例：排序程序的设计与实现

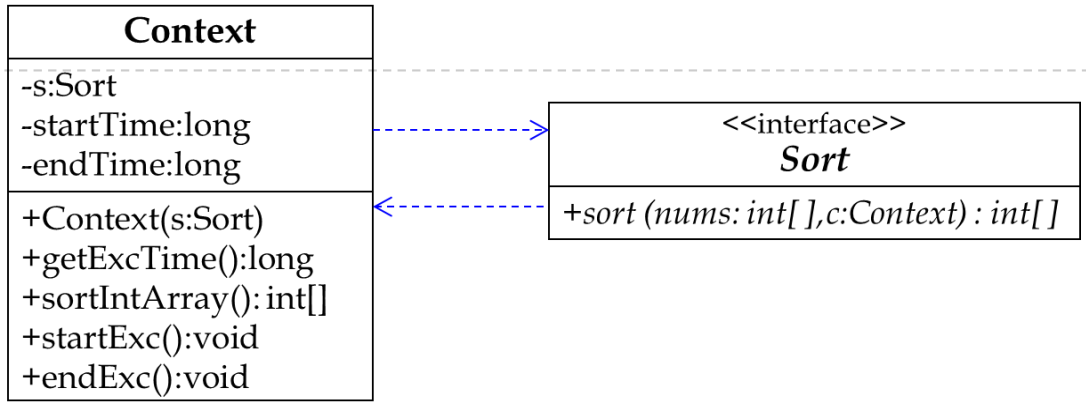
```
public interface Sort {
    int[] sort(int[] num, Context context);
}
```

```
public class HeapSort implements Sort {
    @Override
    public int[] sort(int[] num, Context context) {
        context.startExc();

        //自行实现算法

        context.endExc();
        return num;
    }
}
```

3.1 引例：排序程序的设计与实现




```
public class Context {
    private Sort s;
    private long startTime;
    private long endTime;

    public Context(Sort s) {   this.s = s; }
    public void startExc() { startTime = System.currentTimeMillis(); }
    public void endExc() {  endTime = System.currentTimeMillis();  }

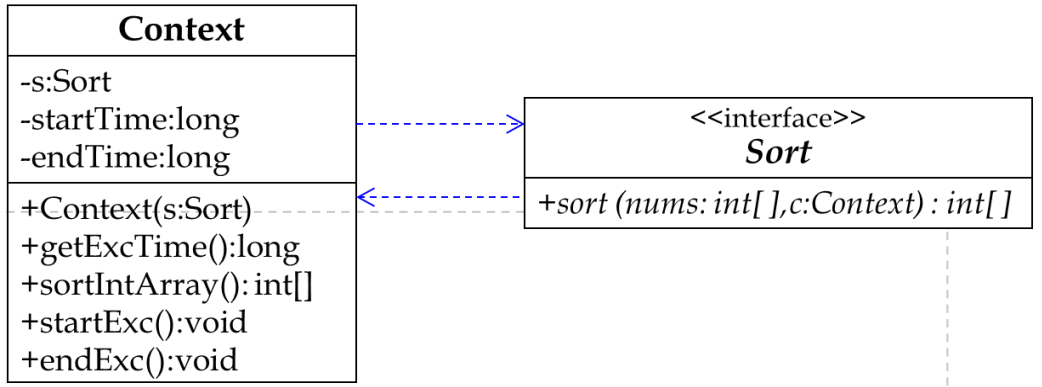
    public long getExcTime() {
        long exeTime = 0;
        exeTime = endTime - startTime;
        return exeTime;
    }
    public int[] sortIntArray(int[] a) {
        return s.sort(a, this);
    }
}
```

这种处理方法加做回调。

回调是一种双向的调用模式，回调的核心就是回调方将本身即this传递给调用方，这样调用方就可以在调用完毕之后再告诉回调方它想要知道的信息。

与Builder模式中的build() 方法思路一致

3.1 引例：排序程序的设计与实现




```java
public class Client {
    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};

        Sort s = new HeapSort();
        Context con = new Context(s);
        arr = con.sortIntArray(arr); // 在该方法内部：return s.sort(a, this);
        System.out.println(Arrays.toString(arr));
        System.out.println(con.getExcTime(););
    }
}
```

```
D:\tools\jdk-17.0.12\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.4.1\lib\idea_rt.jar=9600" -Dfile.
sort by HeadSort
[64, 34, 25, 12, 22, 11, 90]
1006

Process finished with exit code 0
```

3.2 策略模式的原理与实现-策略模式的定义

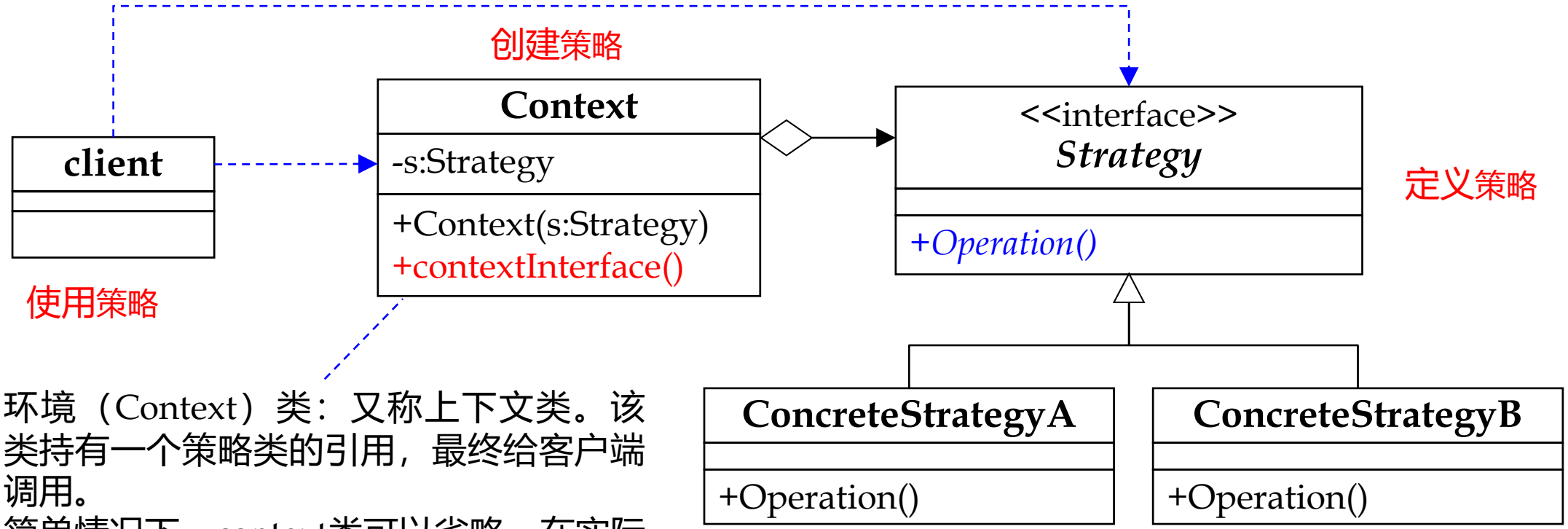
- 策略模式，英文全称是 Strategy Design Pattern。在 GoF 的《设计模式》一书中，它的定义：

> Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it. 定义一族算法类，将每个算法分别封装起来，让它们可以互相替换。策略模式可以使算法的变化独立于使用它们的客户端（这里的客户端代指使用算法的代码）。

3.2 策略模式的原理与实现-策略模式的结构

由 Client 传入策略子类对象

抽象策略（Strategy）类：是一个抽象角色，通常由一个接口或抽象类实现。此角色给出所有的具体策略类所需的接口。



环境（Context）类：又称上下文类。该类持有一个策略类的引用，最终给客户端调用。

简单情况下，context类可以省略。在实际生产过程中，经常以策略工厂替换Context类，产生具体的策略

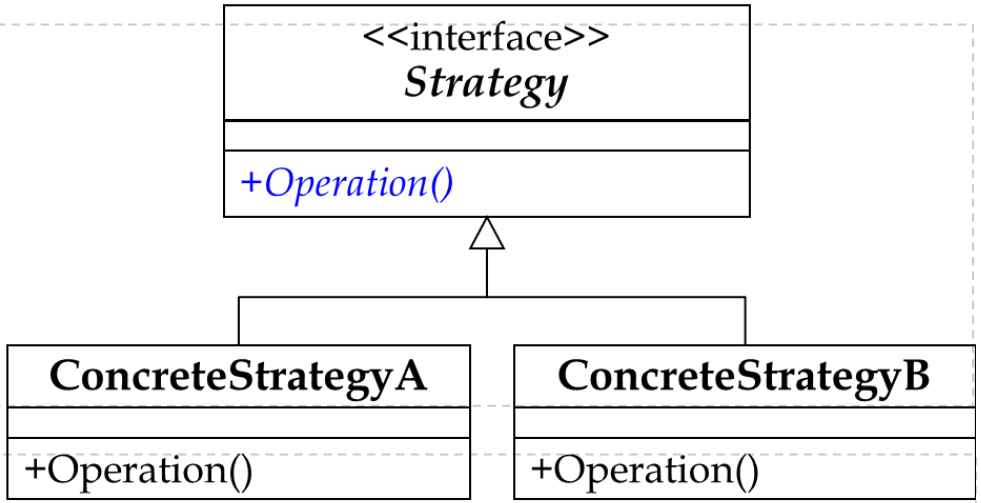
具体策略（Concrete Strategy）类：实现了抽象策略定义的接口，提供具体的算法实现或行为。

3.2 策略模式的原理与实现-策略模式的框架代码

```java
public interface Strategy {
    void operation();
}
```

```java
public class ConcreteStrategyA implements Strategy {
    @Override
    public void operation() {
        //具体的算法...
        System.out.println("ConcreteStrategyA...");
    }
}
```

```java
public class ConcreteStrategyB implements Strategy {
    @Override
    public void operation() {
        //具体的算法...
        System.out.println("ConcreteStrategyB...");
    }
}
```



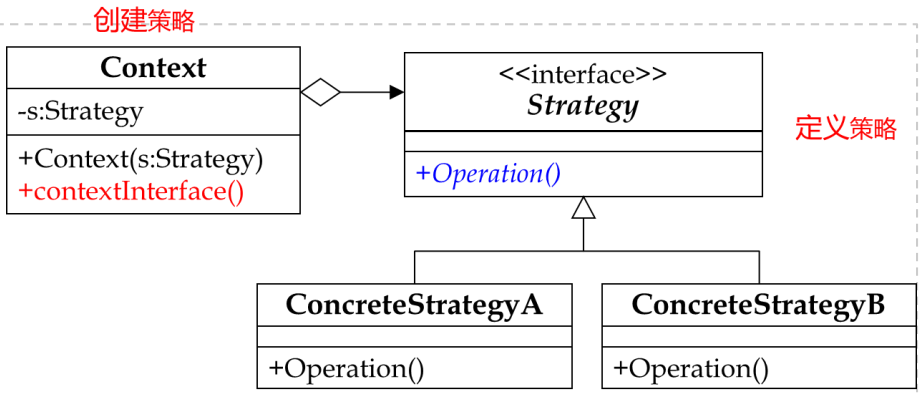
3.2 策略模式的原理与实现-策略模式的框架代码

```
public class Context {
    private Strategy strategy;

    public Context(Strategy strategy) {  // 支持构造函数注入
        this.strategy = strategy;
    }

    public void setStrategy(Strategy strategy) { // 可以动态设置策略
        this.strategy = strategy;
    }

    public void contextInterface() {
        System.out.println("Context contextInterface...");
        strategy.operation();
    }
}
```



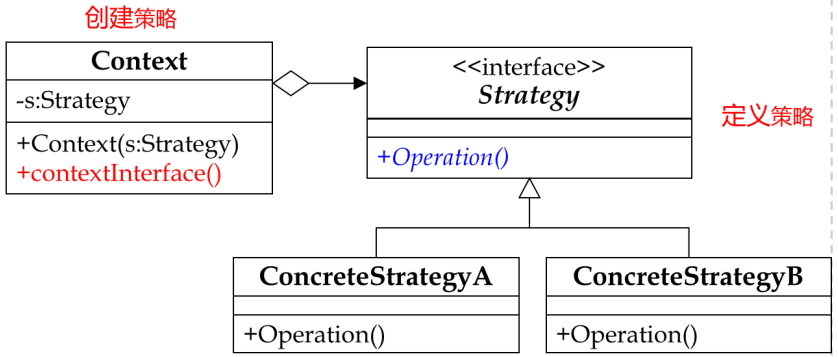
环境（Context）类：又称上下文类。该类持有一个策略类的引用，最终给客户端调用。简单情况下，context 类可以省略。在实际生产过程中，经常以策略工厂替换Context类，产生具体的策略。可以灵活调整

3.2 策略模式的原理与实现-策略模式的框架代码

```
public class Client {
    public static void main(String[] args) {

        Context context= new Context(new ConcreteStrategyA());
        context.contextInterface();
        context.setStrategy(new ConcreteStrategyB());
        context.contextInterface();

    }
}
```



官方提供的结构中，将策略的选择权交给客户端

```
D:\tools\jdk-17.0.9\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.1.1.1\lib\id
Context contextInterface...
ConcreteStrategyA...
Context contextInterface...
ConcreteStrategyB...
```

3.3 课堂实践：利用策略模式避免分支判断

```
// 订单处理的业务逻辑，不同的订单类型，有不同的打折算法
public class OrderService {
    public double discount(Order order) {

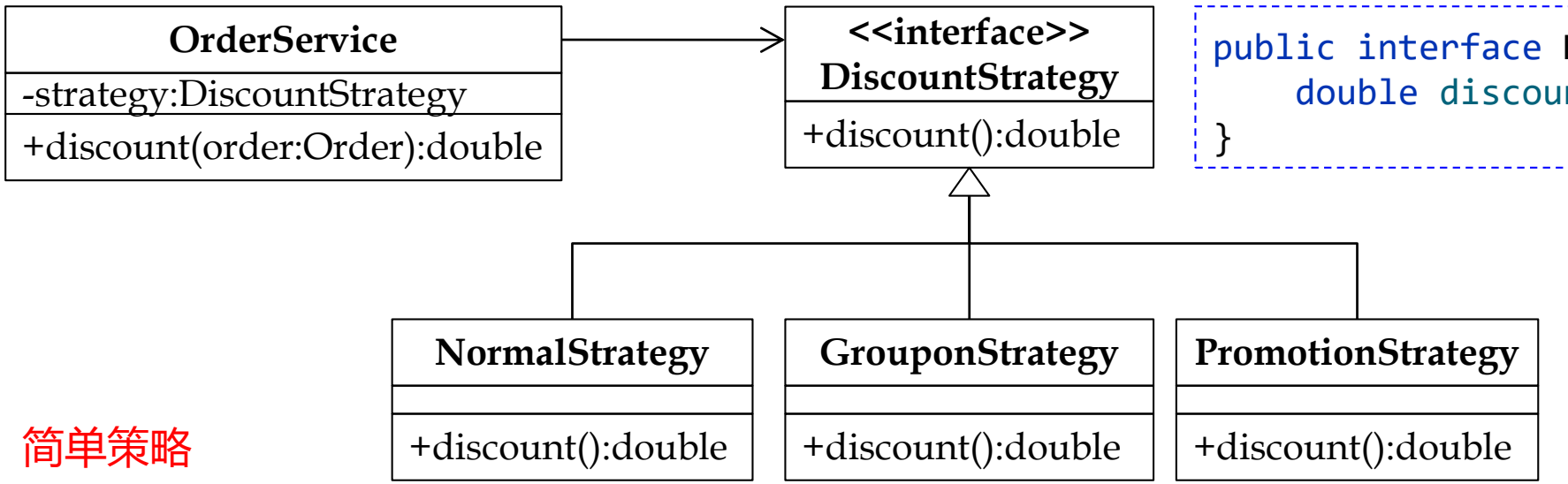
        double discount = 0.0;
        OrderType type = order.getType();

        if (type.equals(OrderType.NORMAL)) { // 普通订单
            //...省略折扣计算算法代码
        } else if (type.equals(OrderType.GROUPON)) { // 团购订单
            //...省略折扣计算算法代码
        } else if (type.equals(OrderType.PROMOTION)) { // 促销订单
            //...省略折扣计算算法代码
        }
        return discount;
    }
}
```

算法分散，应该封装成具体类；
算法具体类应该抽象成层次类

3.3 课堂实践：利用策略模式避免分支判断

算法分散，应该封装成具体类；算法具体类应该抽象成层次类。类图设计如下：




```
public interface DiscountStrategy{
    double discount();
}
```

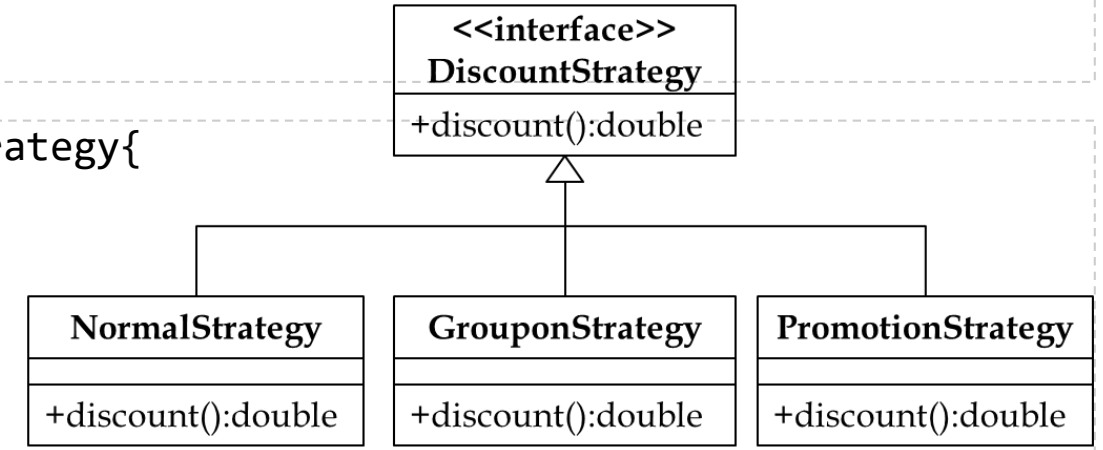
简单策略

此时，OrderService 中的代码就可以基于面向对象的多态特性或“基于接口而非实现编程原则”进行编程

3.3 课堂实践：利用策略模式避免分支判断

```
public class PromotionStrategy implements DiscountStrategy{
    @Override
    public double discount() {
        //...省略折扣计算算法代码
       return 0; //测试用
     }
}
```

```
public class GrouponStrategy implements DiscountStrategy{
    @Override
    public double discount() {
        //...省略折扣计算算法代码
        return 0; //测试用
    }
}
```




```
public class NormalStrategy implements DiscountStrategy{
    @Override
    public double discount() {
        //...省略折扣计算算法代码
        return 0; //测试用
    }
}
```

3.3 课堂实践：利用策略模式避免分支判断

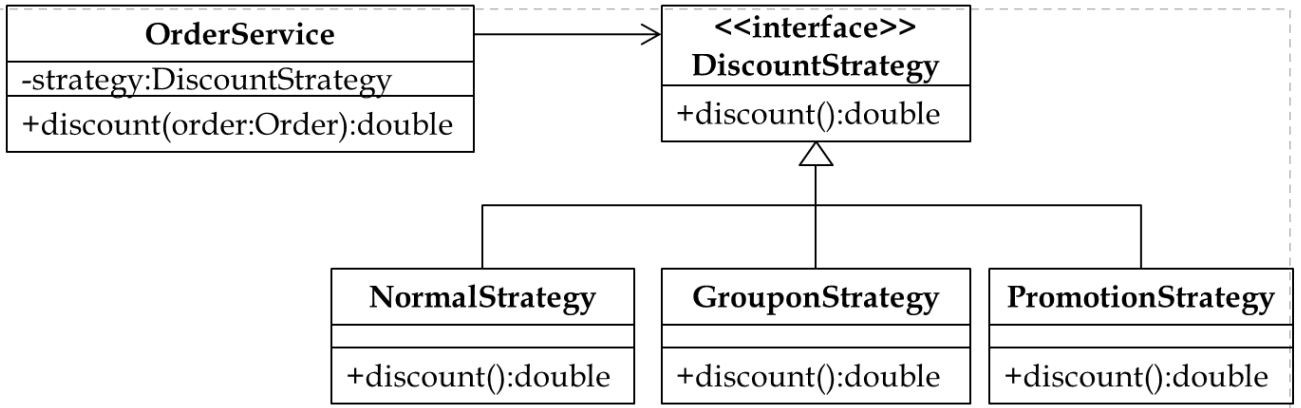
```
public class OrderService {
    private DiscountStrategy strategy;

    public double discount(Order order) {

        double discount = 0.0;
        OrderType type = order.getType();

        if (type.equals(OrderType.NORMAL)) {
            discount = new NormalStrategy().discount();
        } else if (type.equals(OrderType.GROUPON)) {
            discount = new GrouponStrategy().discount();
        } else if (type.equals(OrderType.PROMOTION)) {
            discount = new GrouponStrategy().discount();
        }

        return discount;
    }
}
```



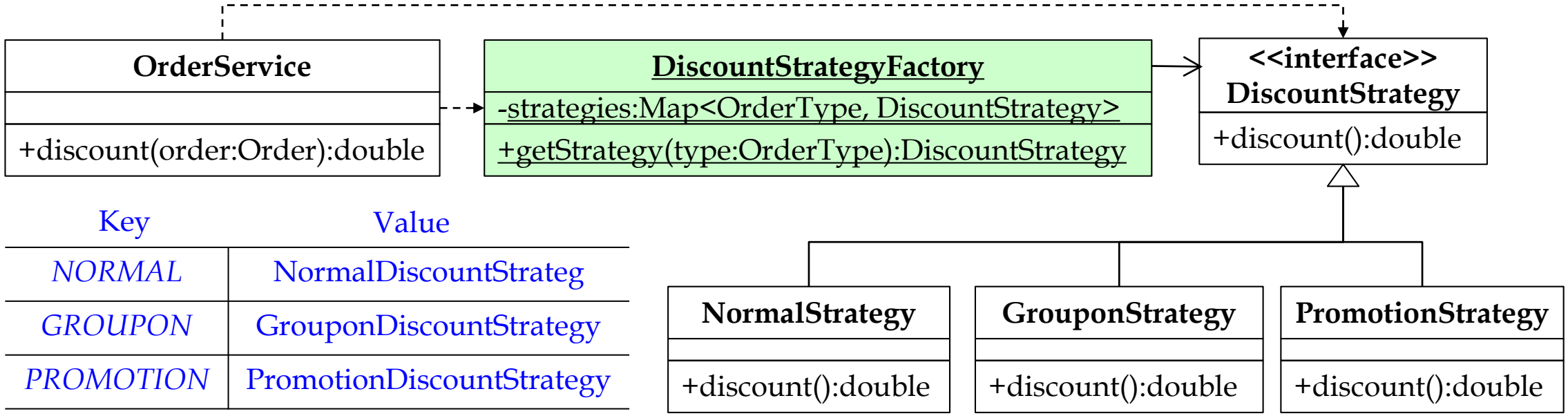
形成层次类，但if..else还是没有解决

转变成了一个对象的生成问题,使用简单工厂模式很容易解决

用简单工厂的查表法或者工厂模式，很容易解决

3.3 课堂实践：利用策略模式避免分支判断

```java
// OrderService
public double discount(Order order) {
    OrderType type = order.getType();
    DiscountStrategy discountStrategy = DiscountStrategyFactory.getStrategy(type);
    return discountStrategy.discount(order);
}
```



Key	Value
NORMAL	NormalDiscountStrateg
GROUPON	GrouponDiscountStrategy
PROMOTION	PromotionDiscountStrategy

3.3 课堂实践：利用策略模式避免分支判断

```
public class DiscountStrategyFactory {
    private static final Map<OrderType, DiscountStrategy> strategies = new HashMap<>();
    static {
        strategies.put(OrderType.NORMAL, new NormalStrategy());
        strategies.put(OrderType.GROUPON, new GrouponStrategy());
        strategies.put(OrderType.PROMOTION, new PromotionStrategy());
    }
    public static DiscountStrategy getDiscountStrategy(OrderType type) {
        return strategies.get(type);
    }
}
```

在策略工厂类中，用 Map 缓存策略，用查表法获得策略

```
public class OrderService {
    public double discount(Order order) {
        OrderType type = order.getType();
        DiscountStrategy discountStrategy = DiscountStrategyFactory.getDiscountStrategy(type);
        return discountStrategy.discount();
    }
}
```

3.3 课堂实践：利用策略模式避免分支判断

```
public class OrderServiceTest {
    @Test
    public void testDiscount(){
        OrderService orderService = new OrderService();
        Order order = new Order();
        order.setType(OrderType.NORMAL);
        double discount = orderService.discount(order);
        System.out.println(discount);
    }
}
```

```
✓ 1 test passed  1 test total, 10 ms
D:\tools\jdk-17.0.9\bin\java.exe ...
NormalStrategy discount
0.0

Process finished with exit code 0
```

3.4 策略模式讨论-策略模式与简单工厂模式的区别

- 策略模式与简单工厂模式在类图上没有多大区别。但二者的核心思想不同。

模式	核心思想	关键区别
策略模式	封装可互换的行为	关注算法或行为的封装和选择，将算法或行为的变化与算法的调用者分离
简单工厂模式	封装对象的创建过程	关注对象的创建过程，将对象的创建与使用分离

3.5 学生练习：诸葛亮的锦囊妙计

- 三国演义中有一个经典故事：赵云保护刘备入吴迎娶孙尚香，诸葛亮担心刘备被扣押，给了赵云三个锦囊(Plan)，每个锦囊中都有对应的行动（action），嘱咐赵云伺机行事。
- 三个锦囊分别是：
  - 贿赂乔国老，走金钱外交，建议用 Monetary 定义此锦囊类
  - 忽悠孙国太，打上层权利牌，以权力施压，逼迫放人，建议用 Pressure 定义此锦囊类
  - 请孙夫人退兵，打感情牌，建议用 Emotion 定义此锦囊类
- 赵云可以抽象为护卫 Guard，伺机行事可以定义为方法 performPlan()
- 请绘制以上故事表达的“使用三个锦囊的类图”，要求满足OCP；基于类图，实现代码，并编写测试类模拟使用三个锦囊的过程。

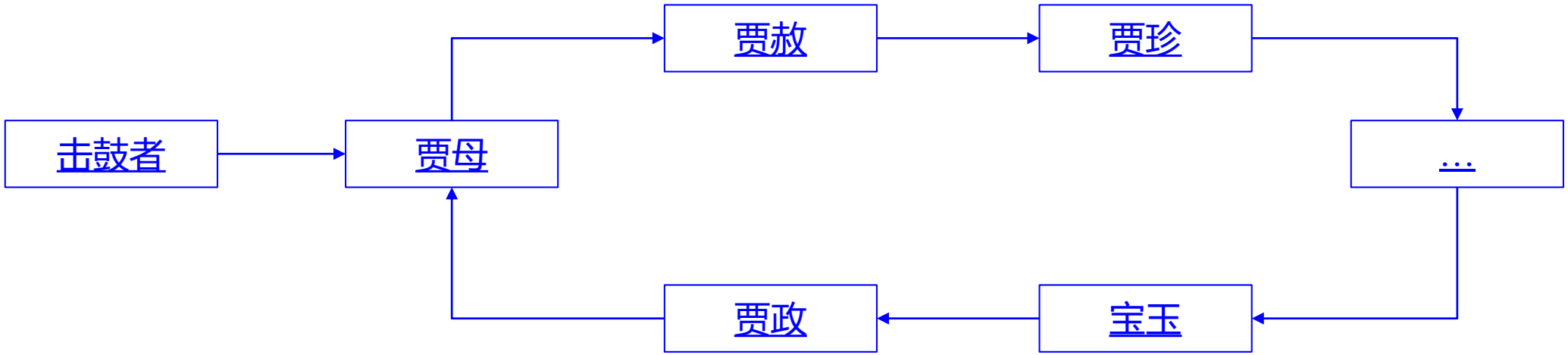
4. 责任链设计模式
Chain of Responsibility Pattern

4.1 从击鼓传花谈起-故事介绍

《红楼梦》第七十五回里生动地描述了贾府里的一场击鼓传花游戏：“贾母坐下、左垂首贾赦、贾珍、贾琏、贾蓉；右垂首贾政、宝玉、贾环、贾兰、团团围坐。….贾母便命折一枝桂花来，命一媳妇在屏后击鼓传花。若花到了谁手中，饮酒一杯……于是先从贾母起，次贾赦，一一接过。鼓声两转，恰恰在贾政手中住了，只得饮了酒。”

- 请根据红楼梦描述的场景，设计击鼓传花这一游戏。

4.1 从击鼓传花谈起-示意图及说明

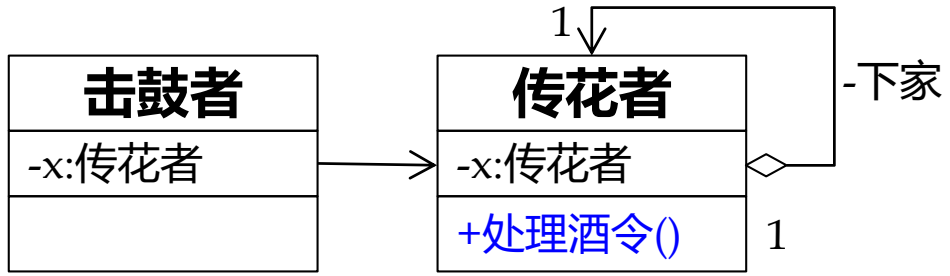


- 每个参加游戏的人在接到传来的花的时候，可选行为只有两个：要么将花向下传，要么执行酒令-喝酒。一个人不能既执行酒令，又向下家传花。

- 当某一个人执行了酒令之后，游戏重新开始，即：游戏的起点是动态的。击鼓的人并不知道最终是哪一个做游戏的人执行酒令

4.1 从击鼓传花谈起-单独考虑击鼓传花系统

- 单独考虑击鼓传花这一独立的游戏系统：那么贾母，贾赦，贾政，贾宝玉等参加游戏的人是一个个“传花者”对象。而击鼓的人便是客户端对象。花代表酒令，是传向处理者的请求。



传花者接收到请求后，可以选择：将请求处理掉，或者将请求传给下家。因此需要持有对下家的引用；

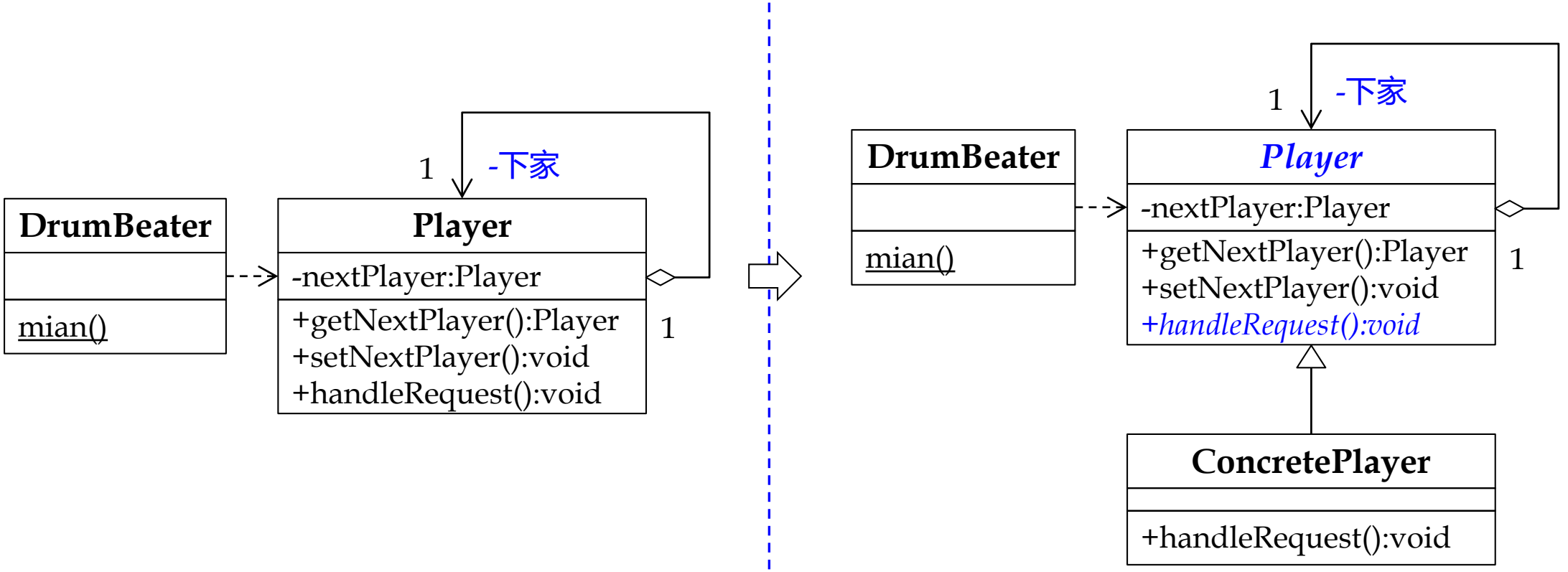


聚合的语义：一个Player知道下家是谁。角色名“下家”，就是指 Player 类中的私有变量 nextPlayer。

Player中的下家，可以通过构造函数注入。本类图中用的是setter注入

4.1 从击鼓传花谈起-考虑可扩展性，形成类层次结构

- 根据我们已有知识，能够想到，为了让设计有一定的扩展性，应该避免直接对具体类编程。



如此设计，可以让不同类型的对象也可以参与进来。

4.1 从击鼓传花谈起-类图中抽象类的代码

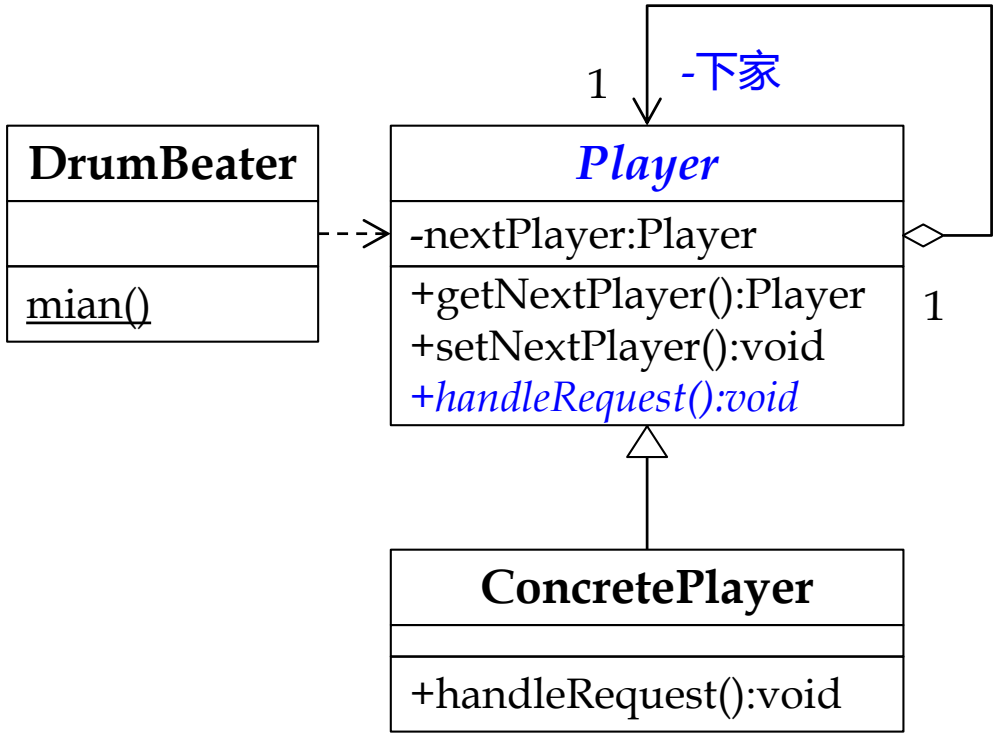
```
public abstract class Player {
    //持有下家的引用
    private Player nextPlayer;

    public Player getNextPlayer() {
        return nextPlayer;
    }

    public void setNextPlayer(Player nextPlayer) {
        this.nextPlayer = nextPlayer;
    }

    //处理酒令
    public abstract void handelRequest();

}
```



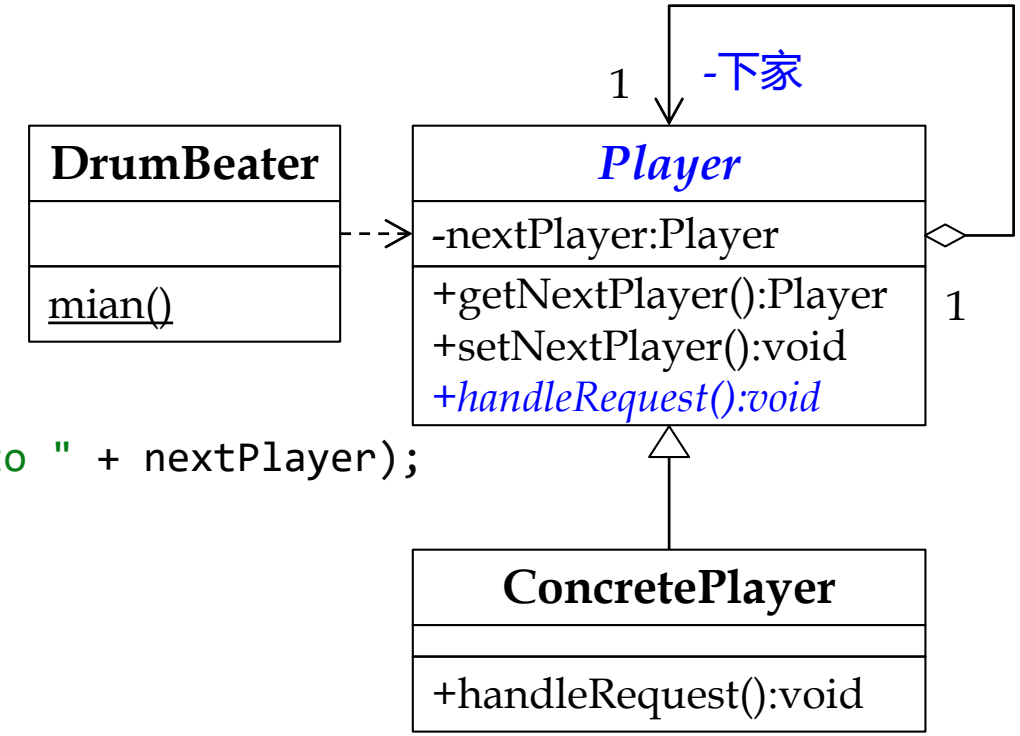
4.1 从击鼓传花谈起-类图中具体类的代码

```
public class ConcretePlayer extends Player {
    @Override
    public void handelRequest() {

        // 获取下家对象
        Player nextPlayer = getNextPlayer();

        // 若有下家，则传递给下家处理
        if (nextPlayer != null) {
            System.out.println("Request is passed to " + nextPlayer);
            // 调用下家的处理方法
            nextPlayer.handelRequest();

        } else {
            // 没有下家，在本对象处理。
            System.out.println("Request is handled by  " + this);
        }
    }
}
```

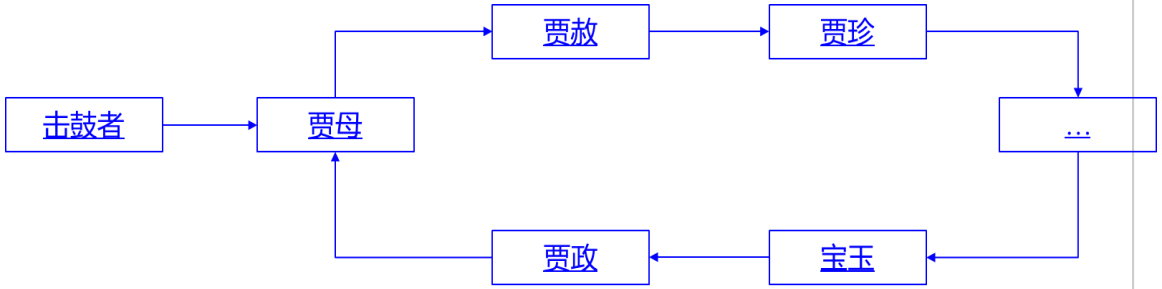


4.1 从击鼓传花谈起-类图中测试类的代码

```
public class DrumBeater {
    public static void main(String[] args) {
        //初始化游戏者
        Player jimu      = new ConcretePlayer();
        Player jiashe    = new ConcretePlayer();
        Player jiazhen   = new ConcretePlayer();
        Player jiabaoyu  = new ConcretePlayer();
        Player jiazheng  = new ConcretePlayer();

        //设置游戏链，为避免死循环，先设置成线
        jimu.setNextPlayer(jiashe);
        jiashe.setNextPlayer(jiazhen);
        jiazhen.setNextPlayer(jiabaoyu);
        jiabaoyu.setNextPlayer(jiazheng);

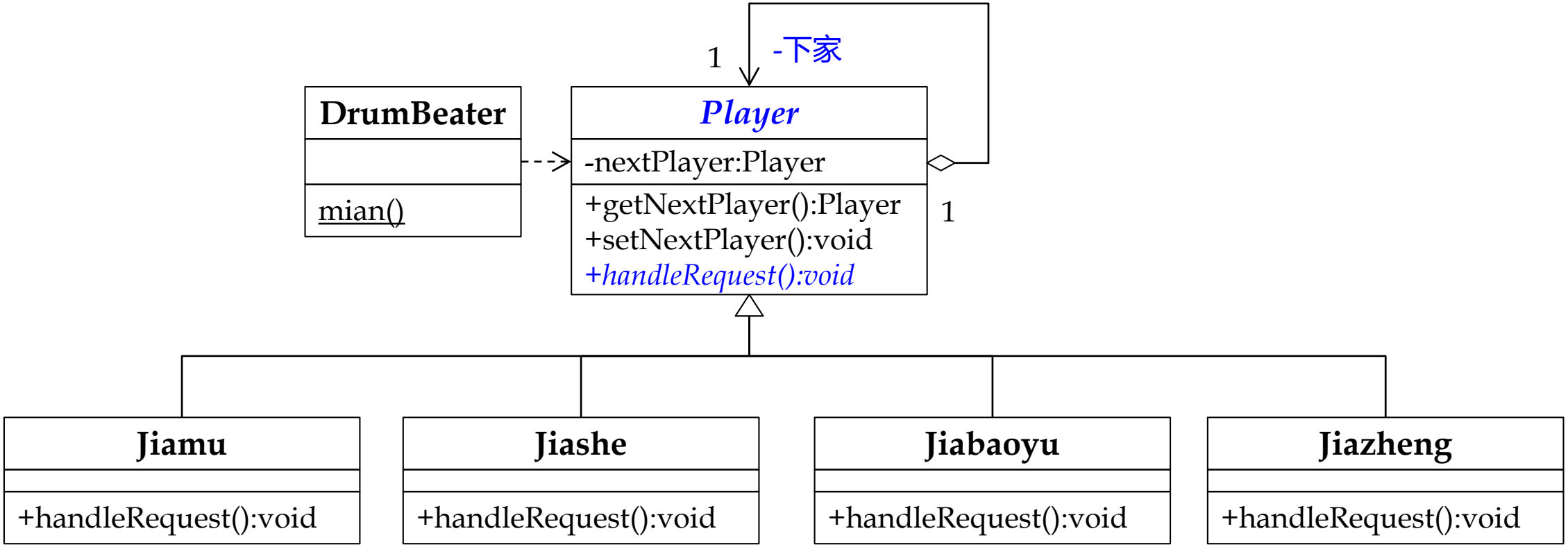
        //从贾母开始传递
        jimu.handelRequest();
    }
}
```




```
D:\tools\jdk-17.0.12\bin\java.exe "-javaagent:C:\Program Fi
Request is passed to drumbeat.ConcretePlayer@448139f0
Request is passed to drumbeat.ConcretePlayer@7cca494b
Request is passed to drumbeat.ConcretePlayer@7ba4f24f
Request is passed to drumbeat.ConcretePlayer@3b9a45b3
The request is handled bydrumbeat.ConcretePlayer@3b9a45b3
```

4.1 从击鼓传花谈起-从“红楼梦”这一大系统中考虑

- 以上是单独考虑击鼓传花这一独立的游戏系统。
- 如果将以上的游戏放到“红楼梦”这一大系统中，贾母，贾赦等游戏的参与者就应该是一个个具体的类代表。



4.1 从击鼓传花谈起-多个类而非一个类的多个对象

- 具体类除了类名发生变化之外，无其他变化。（省略其他的具体类）

```
public class Jiamu extends Player {
    @Override
    public void handelRequest() {
        // 调用从超类继承来的getNextPlayer获取下家对象
         Player nextPlayer = getNextPlayer();

        // 若有下家，则传递给下家处理
         if (nextPlayer != null) {
            System.out.println("Request is passed to " + nextPlayer);
            // 调用下家的处理方法
             nextPlayer.handelRequest();
        } else {
            // 没有下家，在本对象处理。
             System.out.println("Request is handled by  " + this);
        }
    }
}
```

4.1 从击鼓传花谈起-多个类而非一个类的多个对象

- 测试类如下：

变成了不同类的对象

```
public class DrumBeater {
    public static void main(String[] args) {
        //初始化游戏者
        Player jimu      = new Jiamu();
        Player jiashe    = new Jieshe();
        Player jiazhen   = new Jiazhen();
        Player jiabaoyu  = new Jiabaoyu();
        Player jiazheng  = new Jiazheng();
        //设置游戏链，为避免死循环，设置成一条处理线
        jimu.setNextPlayer(jiashe);
        jiashe.setNextPlayer(jiazhen);
        jiazhen.setNextPlayer(jiabaoyu);
        jiabaoyu.setNextPlayer(jiazheng);

        //从贾母开始传递
         jimu.handelRequest();
    }
}
```

```
D:\tools\jdk-17.0.12\bin\java.exe "-javaagent:C:\Program
Request is passed to drumbeat.Jieshe@7699a589
Request is passed to drumbeat.Jiazhen@58372a00
Request is passed to drumbeat.Jiabaoyu@4dd8dc3
Request is passed to drumbeat.Jiazheng@6d03e736
The request is handled by drumbeat.Jiazheng@6d03e736
```

当前逻辑：如果有下家，就交给下家处理。

但此逻辑有问题：每次都是游戏链中排最后那个人要行酒令。应该模拟选择一人行酒令。

如何解决，留给大家作为思考题

4.2 责任链模式-定义

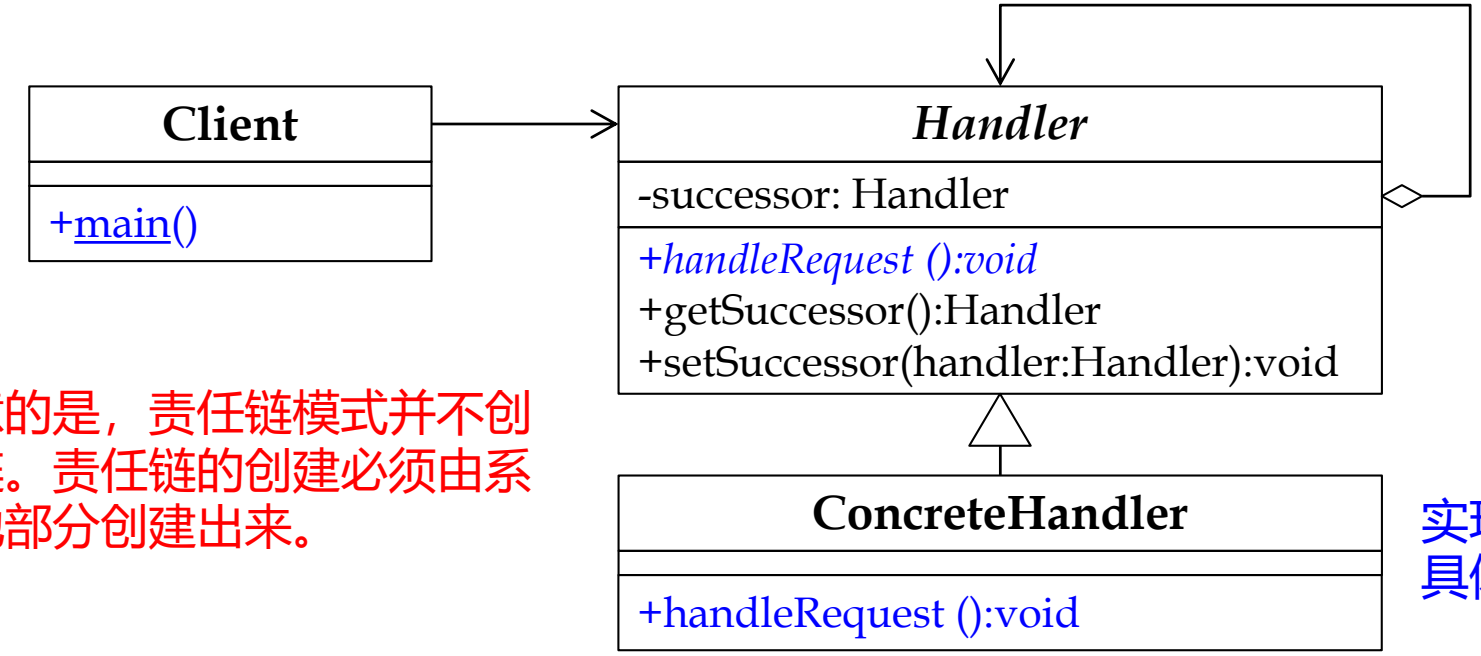
- Chain Of Responsibility Design Pattern，因为翻译不同，称为：责任链模式，职责链模式。在 GoF 的《设计模式》中定义如下：

> Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. Chain the receiving objects and pass the request along the chain until an object handles it.
> 将请求的发送和接收解耦，让多个接收对象都有机会处理这个请求。将这些接收对象串成一条链，并沿着这条链传递这个请求，直到链上的某个接收对象能够处理它为止

4.2 责任链模式-结构

Client 负责创建处理链，并向链中的处理者提交请求。

Handler 定义一个处理请求的接口，并且可以定义一个后继链接。

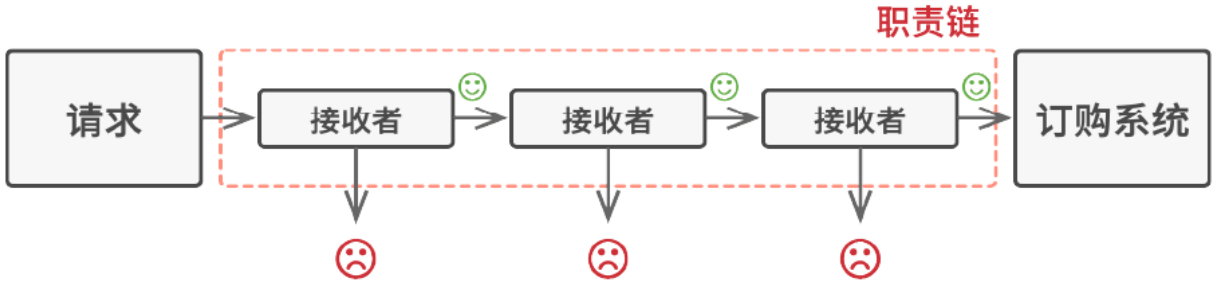


- 值得注意的是，责任链模式并不创建责任链。责任链的创建必须由系统的其他部分创建出来。

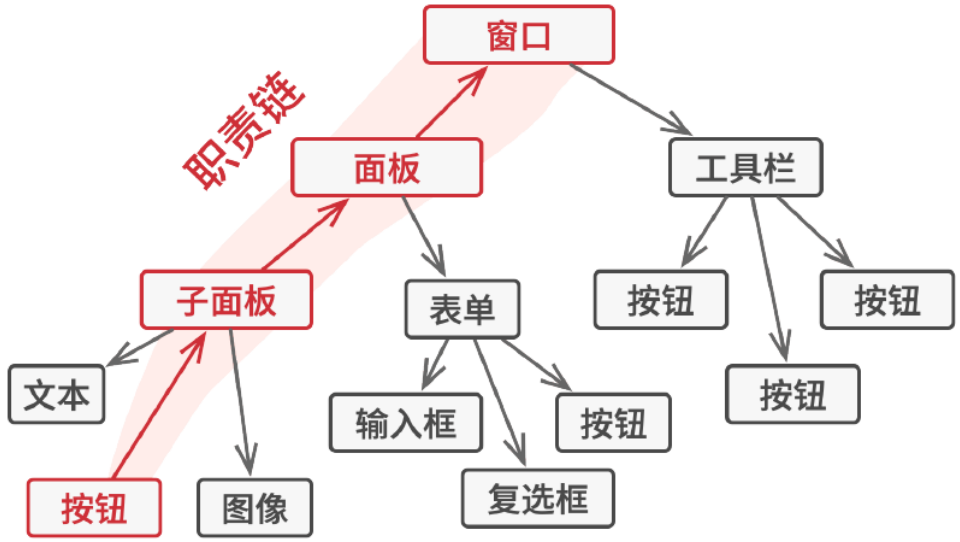
实现抽象处理者的接口，具体处理请求的对象。

- 一个链可以是一条线，可以是一个树，可以是一个环。链的拓扑结构可以是单连通，或者多联通。责任链模式并不制定责任链的拓扑结构。
- 但责任链模式要求在同一时间里，命令只能被传递给下家（或者被处理掉），而不能传给多于一个下家。

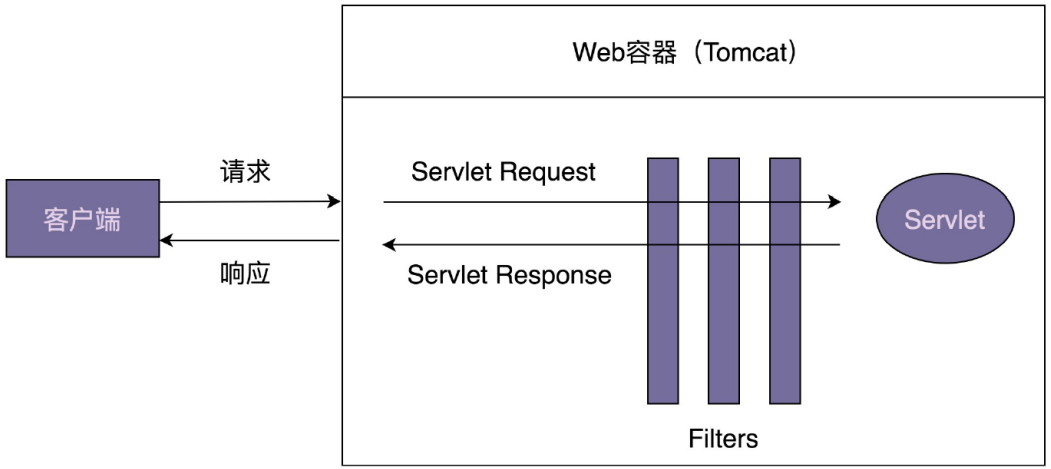
4.2 责任链模式-场景



业务系统的层层审批（例如请假、报销申请审批过程）



页面中的事件处理机制。

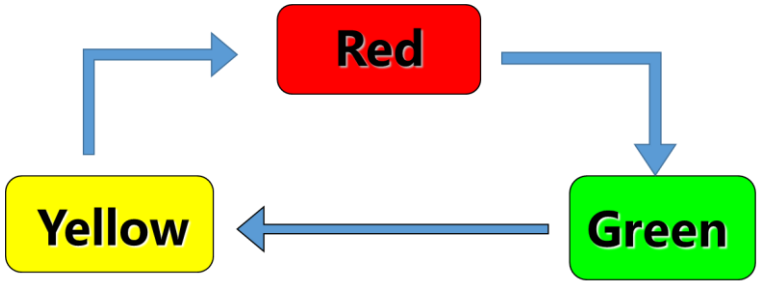


Web容器中的 Servlet Filter 实现鉴权、限流、记录日志、验证参数

5. 状态模式
State Pattern

5.1 交通信号灯系统的设计与实现-需求描述

- 交通信号灯系统中，交通灯的颜色需要在 红灯->绿灯->黄灯->红灯 之间循环转换，但是 不允许绿灯->红灯或黄灯->绿灯等情况。请设计及实现交通信号灯系统。



交通信号状态图

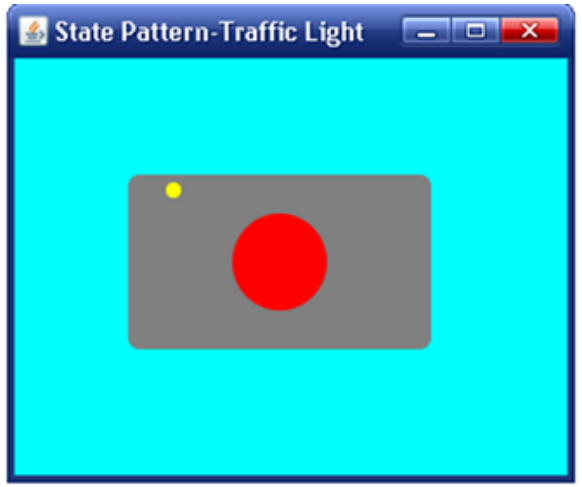


该交通信号灯系统的应用场景

参考资料：https://www.cnblogs.com/FindTheWay/p/14724276.html

5.1 交通信号灯系统的设计与实现-传统的设计思路

- 先不考虑用什么模式，就用传统的设计思路。



  - 首先，我们会很容易想到需要定义一个交通灯颜色的枚举：

```
public enum LightColor {
    Green,Red,Yellow
}
```

  - 然后，定义一个交通灯的类 TrafficLight，在交通灯类中处理颜色转换及相应的业务逻辑

5.1 交通信号灯系统的设计与实现-传统的设计思路

```
public class TrafficLight {
    private LightColor lightColor;
    //初始化为红灯
    public TrafficLight() { lightColor = LightColor.Red; } // 为了省空间，合并代码了
    //信号切换
    public void changeSignal() {
        if (lightColor == LightColor.Red) {
            System.out.println("红灯停");
            lightColor = LightColor.Green;
        } else if (lightColor == LightColor.Green) {
            System.out.println("绿灯行");
            lightColor = LightColor.Yellow;
        } else if (lightColor == LightColor.Yellow) {
            System.out.println("黄灯亮了等一等");
            lightColor = LightColor.Red;
        }
    }
}
```

5.1 交通信号灯系统的设计与实现-传统的设计思路

```
public class Client {
    public static void main(String[] args) {
        TrafficLight light = new TrafficLight();
        light.changeSignal();
        light.changeSignal();
        light.changeSignal();
    }
}
```

运行结果:
红灯停
绿灯行
黄灯亮了等一等

这段代码是完全满足需求的，并且逻辑严谨，调用方式也极其简单，如果需求不变，这或许就是最好的实现方式

5.1 交通信号灯系统的设计与实现-传统的设计的问题

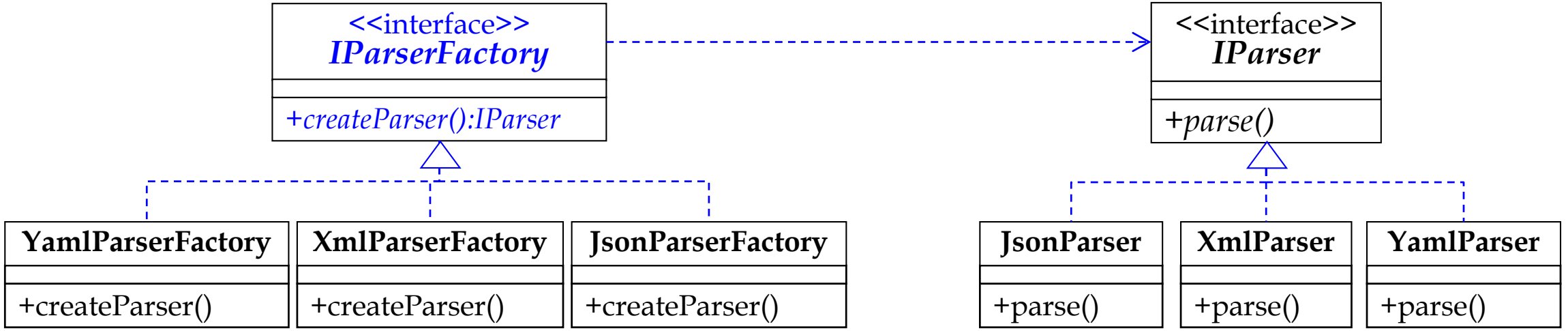
- 但需求不可能不变。

```
public void changeSignal() {
    if (lightColor == LightColor.Red) {
        System.out.println("红灯停");
        lightColor = LightColor.Green;
    } else if (lightColor == LightColor.Green) {
        System.out.println("绿灯行");
        lightColor = LightColor.Yellow;
    } else if (lightColor == LightColor.Yellow) {
        System.out.println("黄灯亮了等一等");
        lightColor = LightColor.Red;
    }
}
```

- 我们很容易发现 TrafficLight 类 代码充斥着 if-else 的条件分支，这就意味着扩展困难，违背了开闭原则

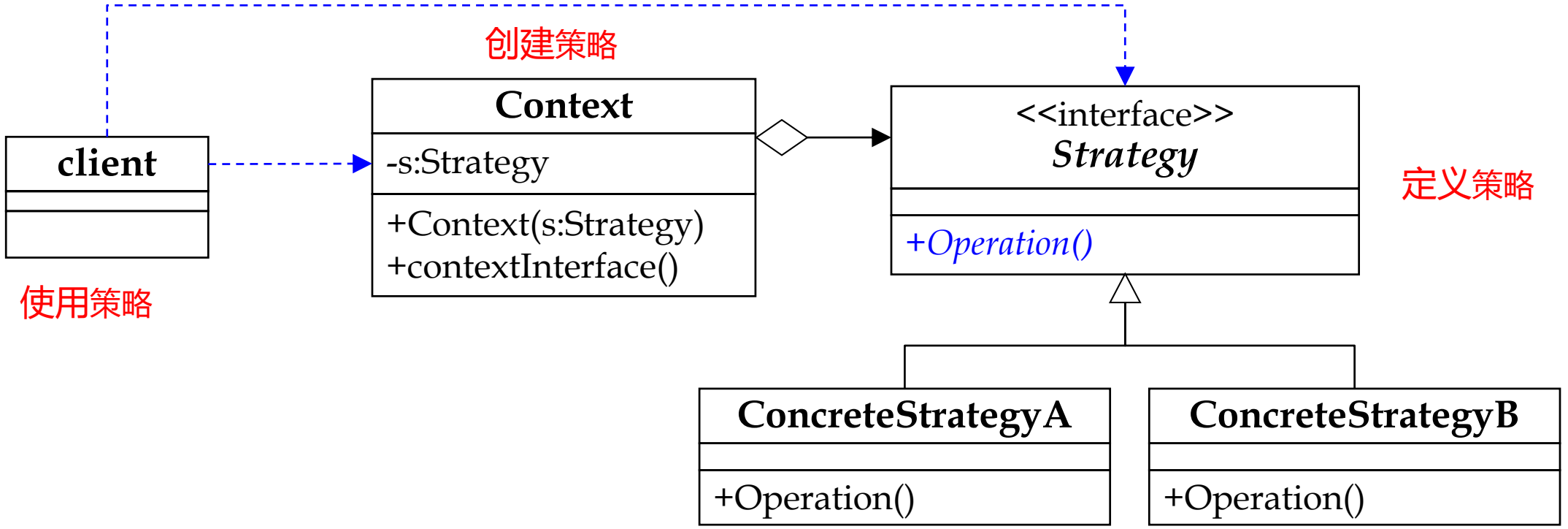
5.1 交通信号灯系统的设计与实现-传统的设计的问题

- 对于解决 if-else 或 switch-case 带来的问题，我们已经相当有经验：
  - 采用工厂方法模式中，抽象出生产具体对象的工厂类解决了 switch-case 的问题



5.1 交通信号灯系统的设计与实现-传统的设计的问题

- 对于解决 if-else 或 switch-case 带来的问题，我们已经相当有经验：
  - 在策略模式中，我们通过将方法抽象成策略类的方式，同样解决了switch-case的问题。



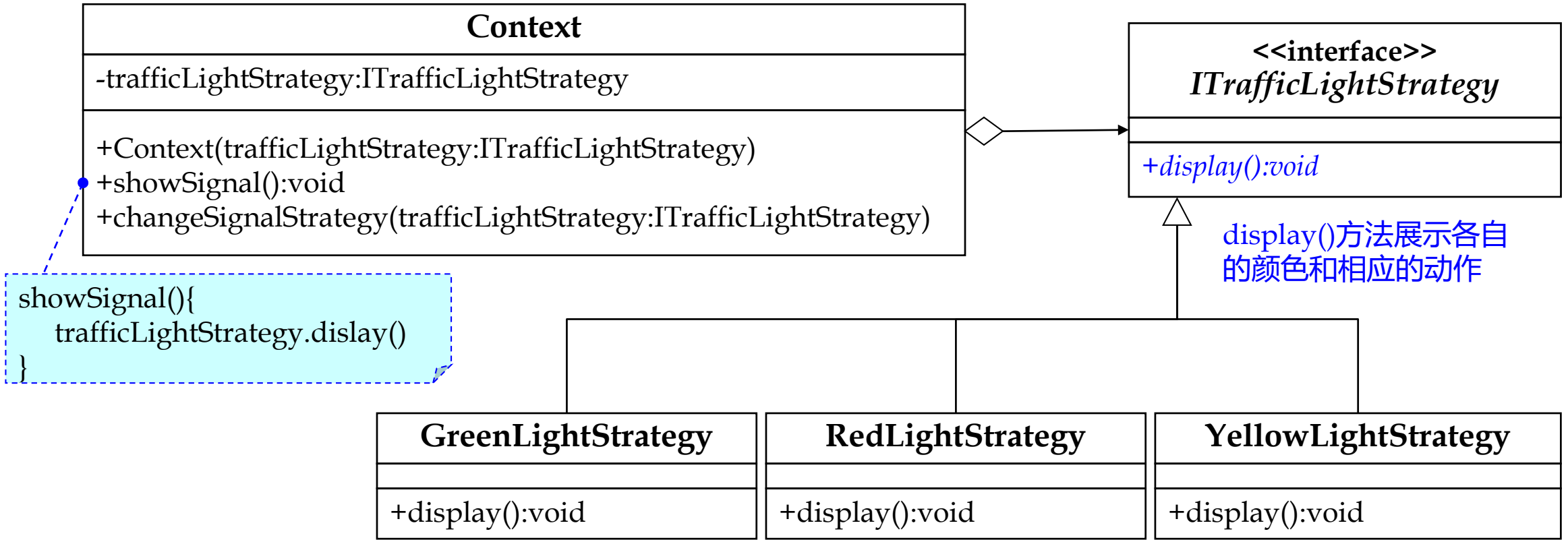
5.1 交通信号灯系统的设计与实现-传统的设计的问题

- 发现这段代码结构跟策略模式极其相似

```
public void changeSignal() {
    if (lightColor == LightColor.Red) {
        System.out.println("红灯停");
        lightColor = LightColor.Green;
    } else if (lightColor == LightColor.Green) {
        System.out.println("绿灯行");
        lightColor = LightColor.Yellow;
    } else if (lightColor == LightColor.Yellow) {
        System.out.println("黄灯亮了等一等");
        lightColor = LightColor.Red;
    }
}
```

- 不妨用策略模式改造一下，策略模式至少能解决 if-else 或 switch-case 的问题

5.1 交通信号灯系统的设计与实现-采用策略模式设计尝试

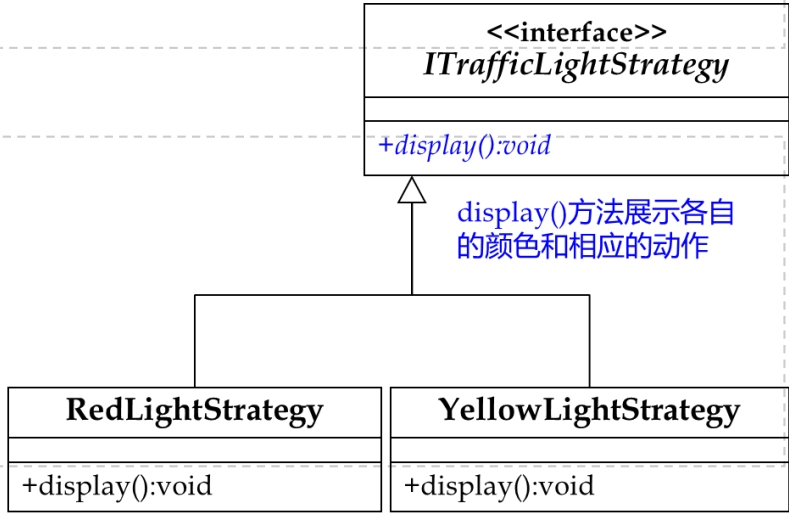


策略形成了层次类，满足OCP；具体类满足单一职责：在做自己最应该做的基础的事情。环境类：引用策略。

5.1 交通信号灯系统的设计与实现-采用策略模式设计尝试

```java
public interface ITrafficLightStrategy {
    void display();
}
```

```java
public class RedLightStrategy implements ITrafficLightStrategy {
    @Override
    public void display() {
        System.out.println("红灯停");
    }
}
```




```java
public class GreenLightStrategy implements ITrafficLightStrategy {
    @Override
    public void display() {
        System.out.println("绿灯行");
    }
}
```

5.1 交通信号灯系统的设计与实现-采用策略模式设计尝试

```
public class Context {
    private ITrafficLightStrategy trafficLightStrategy;

    public TrafficLight(ITrafficLightStrategy trafficLightStrategy) {
        this.signalStrategy = signalStrategy;
    }

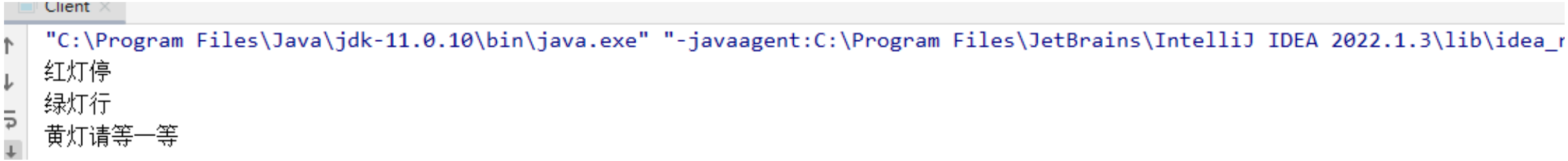
    public void showSignal() {
        if (signalStrategy != null) {
            signalStrategy.displaySignal();
        }
    }

    public void changeSignalStrategy (ITrafficLightStrategy trafficLightStrategy) {
        this.signalStrategy = signalStrategy;
    }
}
```

Context
-trafficLightStrategy:ITrafficLightStrategy
+Context(trafficLightStrategy:ITrafficLightStrategy) +showSignal():void +changeSignalStrategy(trafficLightStrategy:ITrafficLightStrategy)

5.1 交通信号灯系统的设计与实现-采用策略模式设计尝试

```
public class Client {
    public static void main(String[] args) {
        Context context = new Context(new RedLightStrategy());
        context.showSignal();
        context.changeSignalStrategy(new GreenLightStrategy());
        context.showSignal();
        context.changeSignalStrategy(new YellowLightStrategy());
        context.showSignal();
    }
}
```

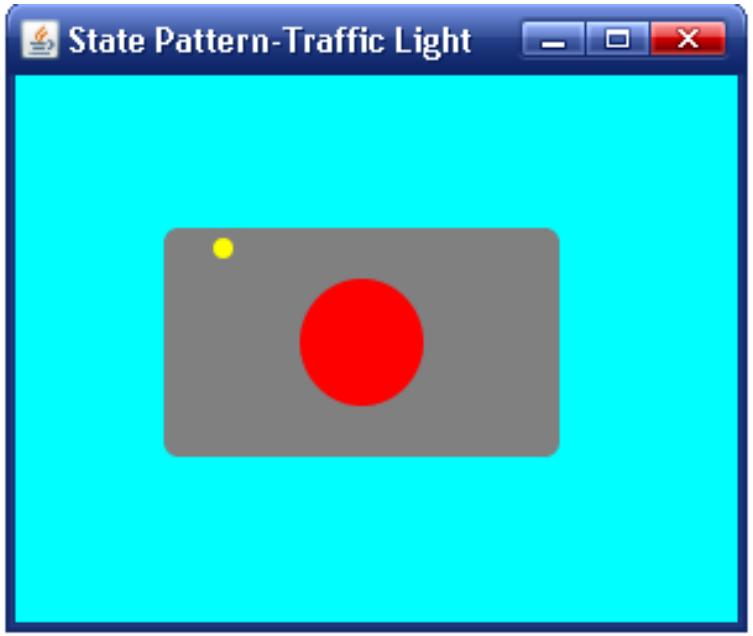


发现：灯颜色的切换交给调用者Client来实现，是非常危险的事情。希望灯颜色切换是由内部一套固定机制控制，而不是调用方来决定，如果用户想换什么颜色就换什么颜色，交通规则岂不乱套了？

显然，策略模式是不满足需求的，我们其实希望 context.changeSignalStrategy() 这个动作，由系统自己内部完成。

5.1 交通信号灯系统的设计与实现-设计反思和修改

- 既然不满足需求，那么问题到底出在哪？





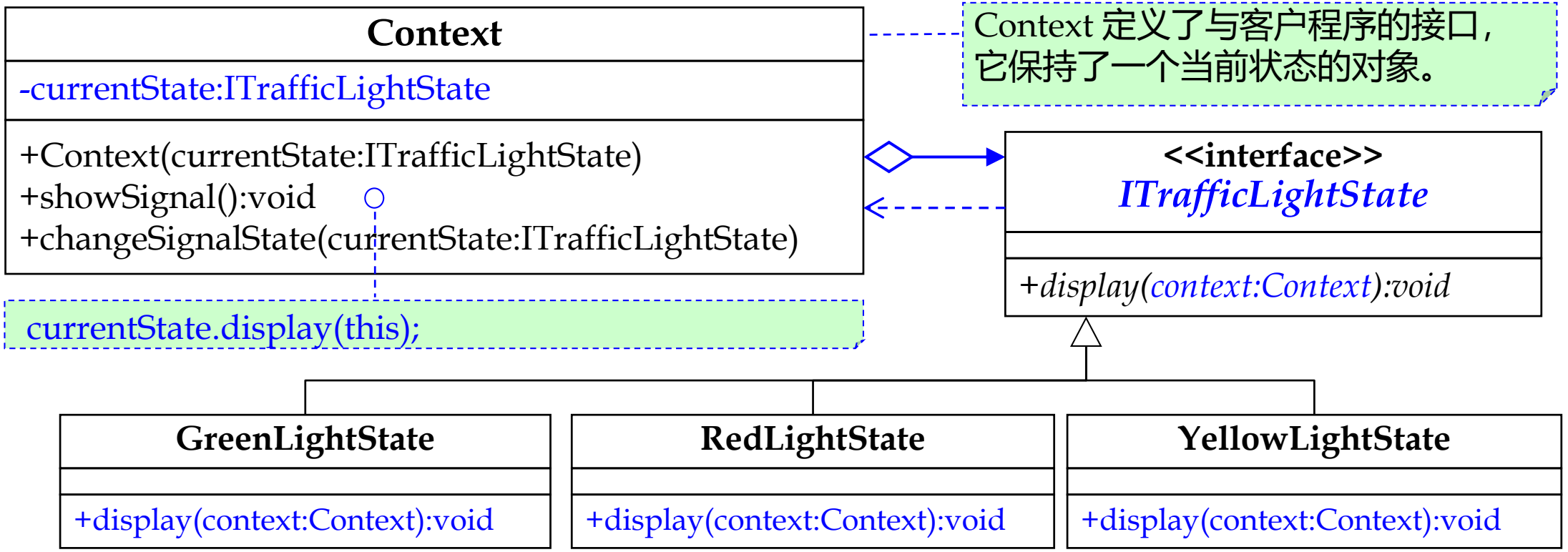
或许我们的思路一开始就出现了偏差，交通灯能换颜色吗？

显然是不能的，因为每个灯的颜色是固定的，我们所谓的换颜色，实际上换的是灯

5.1 交通信号灯系统的设计与实现-设计反思和修改

- 采用工厂方法模式来创造不同颜色的灯？
  - 显然不合适，三个灯一开始就在那里，只是循环切换而已，不存在创建的过程
- 换一种思路：每种颜色的灯表达一种状态
  - 每种灯，也就是每个状态对应需要处理的行为动作
  - 每个状态内部切换到下一个状态，而不是由客户端来进行切换。

5.1 交通信号灯系统的设计与实现-设计反思和修改




```
//在具体的状态子类中告诉环境对象Context，下一个状态是谁。
System.out.println("黄灯停一停");
context.changeCurrentSignal(new RedLightState());
```

5.1 交通信号灯系统的设计与实现-设计反思和修改

```
public interface ITrafficLightState {
    void display(Context context); // 反向关联到Context，取得系统的上下文环境
}
```

```
public class RedLightState implements ITrafficLightState{
    @Override
    public void display(Context context) {
        System.out.println(“红灯停”);
        context.changeCurrentSignal(new GreenLightState()); //红灯之后切换到绿灯
    }
}
```

```
public class GreenLightState implements ITrafficLightState {
    @Override
    public void display(Context context) {
        System.out.println(“绿灯行”);
        context.changeCurrentSignal(new YellowLightState()); //绿灯之后切换到黄灯
    }
}
```

YellowLightState 类与以上一致，逻辑上，黄灯亮后，切换到红灯。页面所限，略...

5.1 交通信号灯系统的设计与实现-设计反思和修改

```
public class Context {
    private ITrafficLightState currentState;

    public Context() {
        this.currentState = new RedLightState();
    }

    public void showSignal() {
        if (currentState != null) {
            currentState.display(this); // this表示当前context对象,具体是谁，需要看客户端
        }
    }

    public void changeCurrentSignal(ITrafficLightState currentState) {
        this.currentState = currentState;
    }
}
```

5.1 交通信号灯系统的设计与实现-设计反思和修改

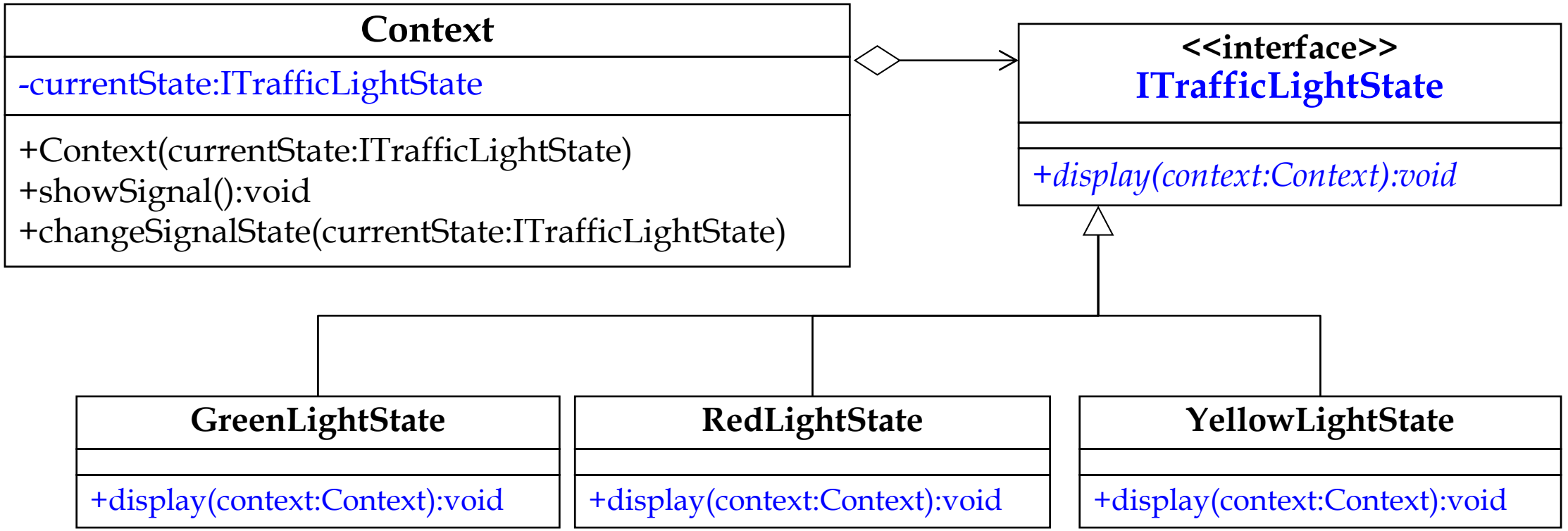
```
public class Client {
    public static void main(String[] args) {
        Context context = new Context();
        context.showSignal();
        context.showSignal();
        context.showSignal();
    }
}
```

除了类名和方法名变了，代码跟策略模式几乎一模一样，但含义却是天差地远。

- 将方法抽象不同的状态，并为每个状态提供处理该状态下对应行为的接口方法，而不是直接提供具体行为的接口方法。
- 参数有所不同，TrafficLightState 中需要持有对 Context 的引用，因为需要在具体的状态类中处理 Context 的状态转移。

改造后的代码满足需求，调用变得简单，状态的转移回归了主权，这就是状态模式

5.1 交通信号灯系统的设计与实现-状态模式

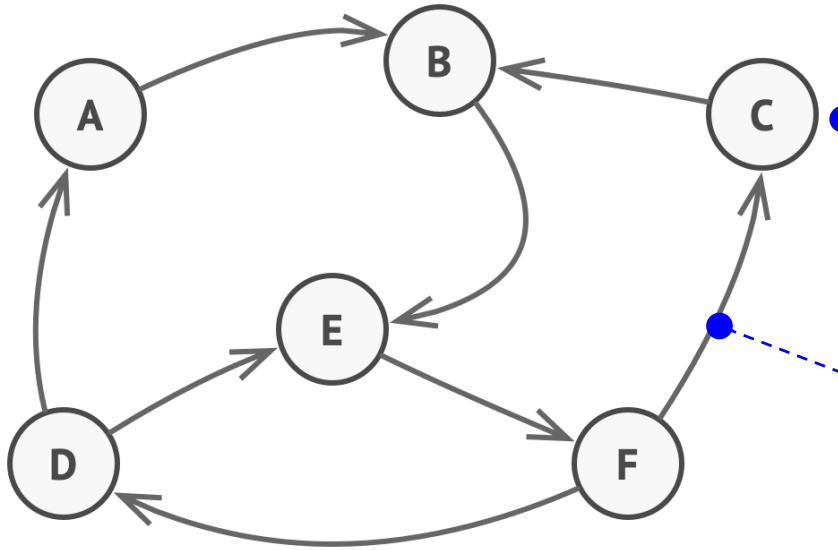



```
//在具体的状态子类中告诉环境对象Context，下一个状态是谁。
System.out.println("黄灯停一停");
context.changeCurrentSignal(new RedLightState());
```

5.2 状态模式-定义

允许一个对象在其内部状态改变时改变它的行为。状态模式与有限状态机的概念紧密相关。其主要思想是程序在任意时刻仅可处于几种有限的状态中。

在任何一个特定状态中，程序的行为都不相同，且可瞬间从一个状态切换到另一个状态。



状态。例如A、B、C、D、E、F都是一种状态

转移：例如，从A->B, 从B->E、从E->F，从F->D或者F->C 等等，都是转移

有限状态机

根据当前状态，程序可能会切换到另外一种状态，也可能会保持当前状态不变。这些数量有限且预先定义的状态切换规则被称为转移。

5.2 状态模式-定义

例如：你有一个文档 Document 类。文档可能会处于草稿 Draft 、审阅中 Moderation 和已发布 Published 三种状态中的一种。文档的 publish 发布方法在不同状态下的行为略有不同：

处于审阅中状态时，如果当前用户是管理员，它会公开发布文档。



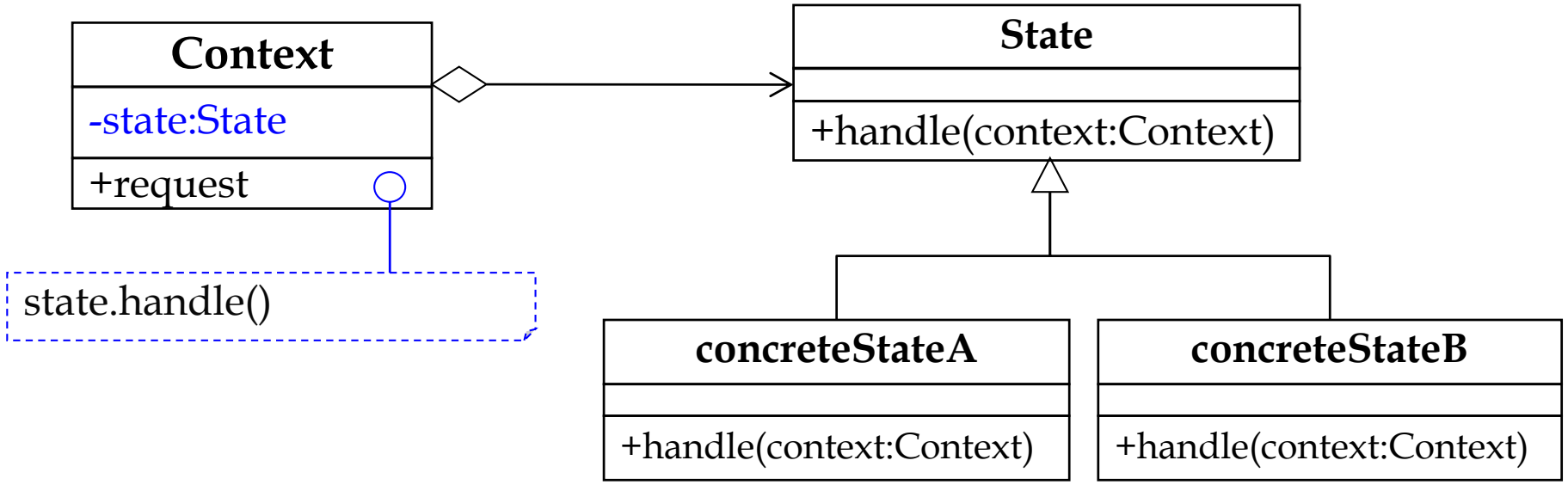
处于草稿状态时，它会将文档转移到审阅中状态

处于已发布状态时，它不会进行任何操作。

5.2 状态模式-结构

Context，上下文环境，定义客户程序需要的接口，并维护一个具体状态角色的实例，将与状态相关的操作委托给当前的ConcreteState 对象来处理；状态的切换是由上下文对象来控制

State：抽象状态类。定义了状态接口，它的各个子类封装了在各种不同状态下的行为

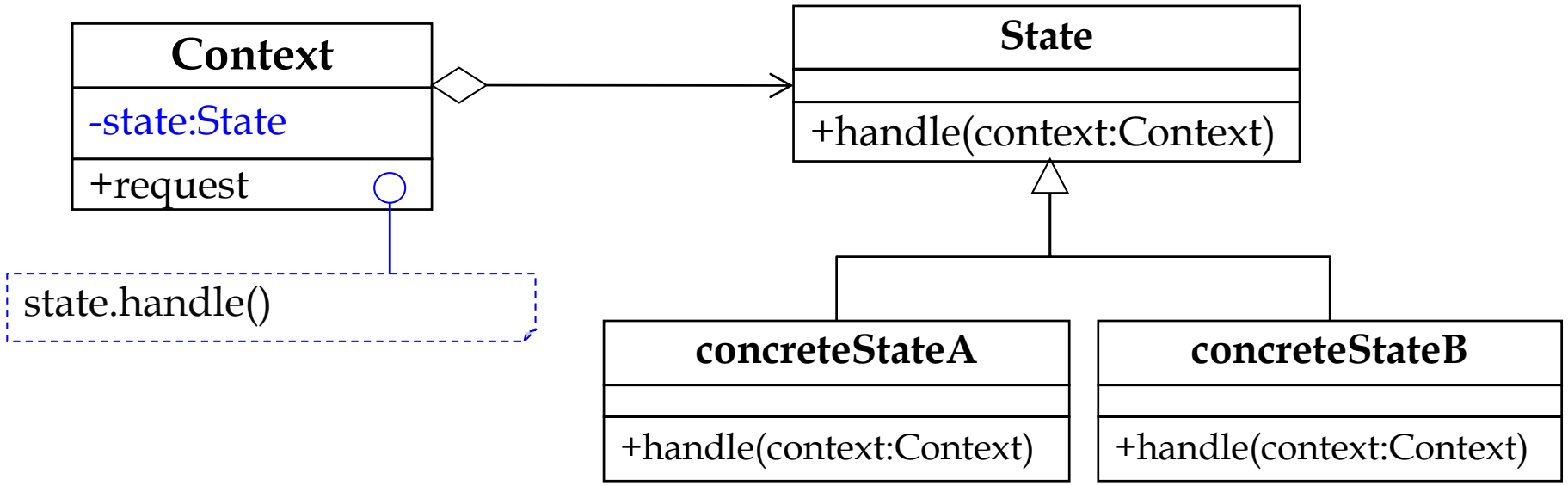


状态模式的关键在于将状态的切换和行为的实现分离开来，并且状态的转换由上下文对象控制，而不是交给client决定

ConcreteState子类：具体的状态类。封装了在各种不同状态下的行为

5.2 状态模式-结构

下图为 GOF 给出的状态模式的标准结构。此结构中并未表达出 State 与 Context 的依赖关系。



状态模式可以理解为，状态的变化在具体类中完成。例如，在红灯类中设定下一个状态是绿灯。

5.2 状态模式-优缺点

- 优点
  - 解决switch-case、if-else带来的难以维护的问题，这个很明显，没什么好说的；
  - 结构清晰，提高了扩展性，不难发现，Context类简洁清晰了，扩展时，几乎不用改变，而且每个状态子类也简洁清晰了，扩展时也只需要极少的改变。
- 缺点
  - 随着状态的扩展，状态类数量会增多，老生常谈，几乎所有解决类似问题的设计模式都存在这个缺点；
  - 增加了系统复杂度，使用不当将会导致逻辑的混乱，状态类增多，且涉及到状态的转移；
  - 不完全满足开闭原则，因为扩展时，除了新增或删除对应的状态子类外，还需要修改涉及到的相应状态转移的其它状态类。

5.3 状态模式和策略模式的异同点

	策略模式 Strategy Pattern	状态模式 State Pattern
相同点	本质上两种模式都是做同一件事情：去耦合。就是把干什么和怎么干分开，互不依赖。两者都用于解决对象行为的变化和动态切换的问题，类图一致。	

差异点	强调可以互换的算法；	强调改变对象内部的状态来帮助控制自己的行为
	用户Client直接与具体算法交互，决定算法的替换，需要了解算法本身；	状态是对象内部流转，用户不会直接跟状态交互，不需要了解状态本身；
	策略类一般不需要持有Context的引用。即便持有，也不做策略切换，只获取数据	状态类需要持有Context的引用，用来实现状态转移。

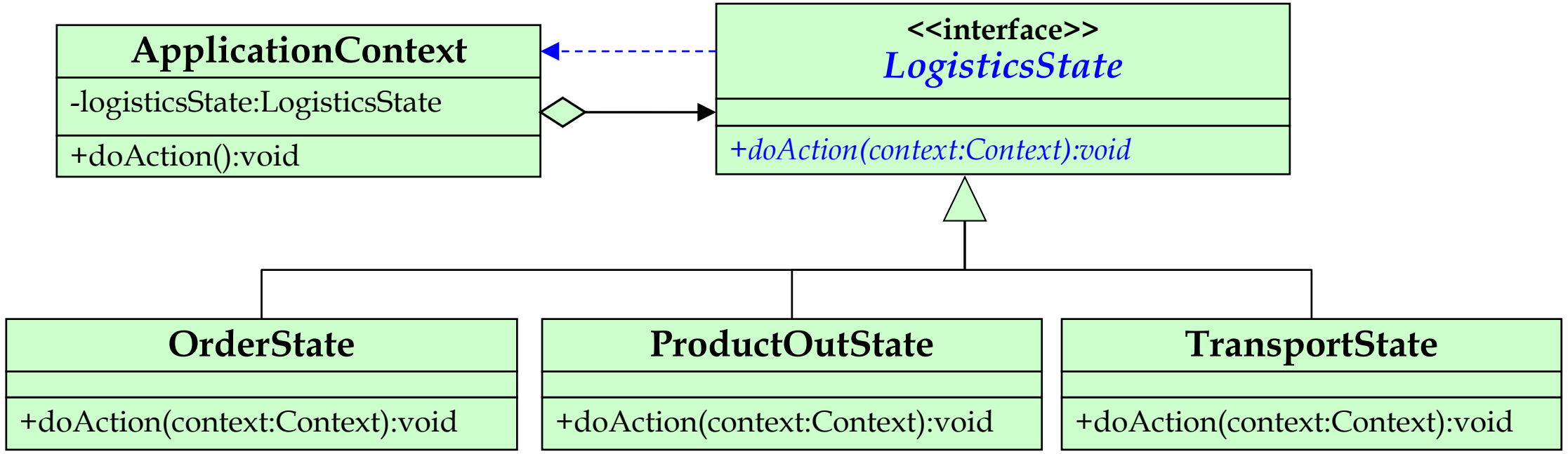
5.4 课堂练习-物流系统状态控制组件的设计与实现

- 在物流系统中，存在很多状态，例如接单、出库、运输、收货、评价等等。而订单在每个不同的状态下的操作可能都不一样，例如在接单状态下，商家就需要通知仓库拣货，通知用户等等操作。
- 要求绘制物流系统的状态控制组件的类图。
- 给出如下名词列表：
  - 状态LogisticsState，接单OrderState ，出库ProductOutState，运输TransportState，其他略…
  - 环境类ApplicationContext

5.4 课堂练习-物流系统状态控制组件的设计与实现

ApplicationContext 是物流的环境类，负责在流程中保持并切换状态

抽象状态类。定义了状态接口，它的各个子类封装了在各种不同状态下的行为



具体的状态类。封装了在各种不同状态下的行为。依次类推，可以建立任意多个状态类

5.4 课堂练习-物流系统状态控制组件的设计与实现

```
public interface LogisticsState {
    void doAction(JdLogistics context);
}
```

```
public class OrderState implements LogisticsState {
    @Override
    public void doAction(ApplicationContext context) {
        System.out.println("商家已经接单，正在处理中...");
        context.setLogisticsState(new ProductOutState()); // 设置下一个状态
    }
}
```

```
public class ProductOutState implements LogisticsState {
    @Override
    public void doAction(ApplicationContext context) {
        System.out.println("商品已经出库...");
        context.setLogisticsState(new TransportState()); // 设置下一个状态
    }
}
```

5.4 课堂练习-物流系统状态控制组件的设计与实现

```
public class ApplicationContext {
    private LogisticsState logisticsState;
    public void setLogisticsState(LogisticsState logisticsState) {
        this.logisticsState = logisticsState;
    }
    public LogisticsState getLogisticsState() {
        return logisticsState;
    }
    public void doAction(){
        Objects.requireNonNull(logisticsState);
        logisticsState.doAction(this);
    }
}
```

5.4 课堂练习-物流系统状态控制组件的设计与实现

```
public class StateClient {
    public static void main(String[] args) {
         //状态的保持与切换者
        ApplicationContext  context = new ApplicationContext ();

        //接单状态
        context.doAction();

        //出库状态
        context.doAction();

        //运输状态
        context.doAction();
    }
}
```

```
商家已经接单，正在处理中...
商品已经出库...
商品正在运往天津分发中心
```

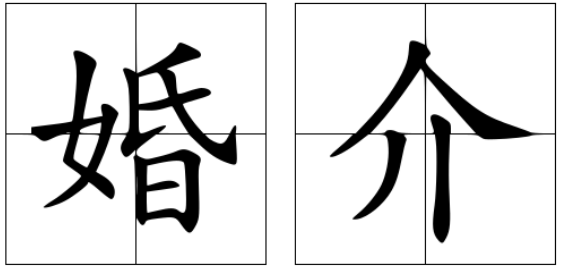
6. 中介模式
Mediator Pattern

6.1 生活中的中介







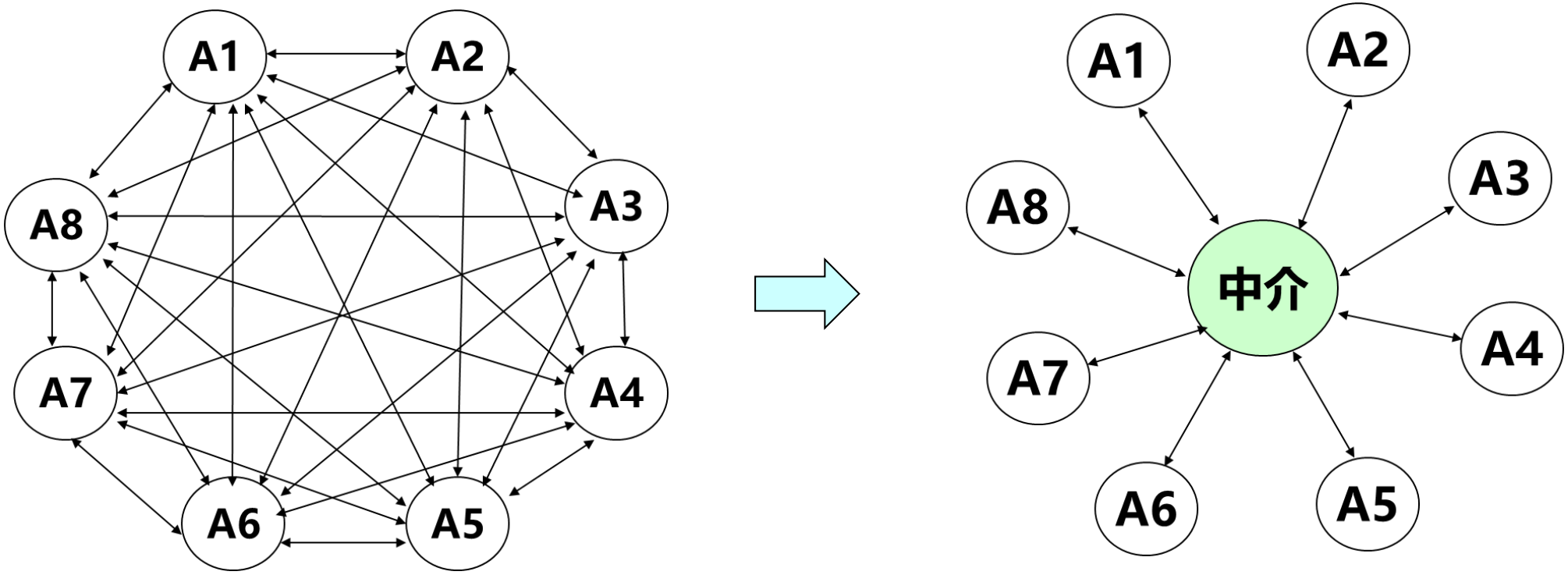








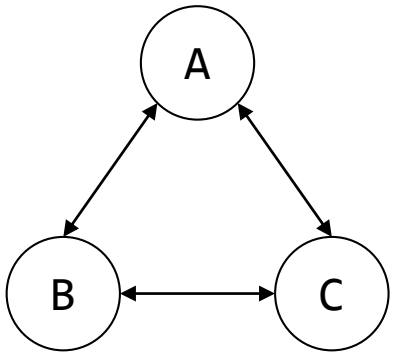
6.1 生活中的中介



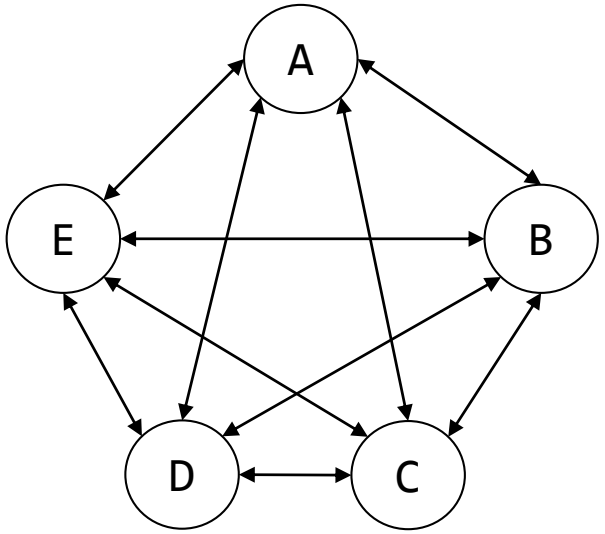
例如，在二手汽车交易市场，由于交易双方在交易前互不了解，因此可能寻求一位具有中间立场并且具有一定素质的人或者公司，来对交易进行经济或者信誉上的的担保，以推动买卖顺利进行。

6.2 软件设计领域也需要中介

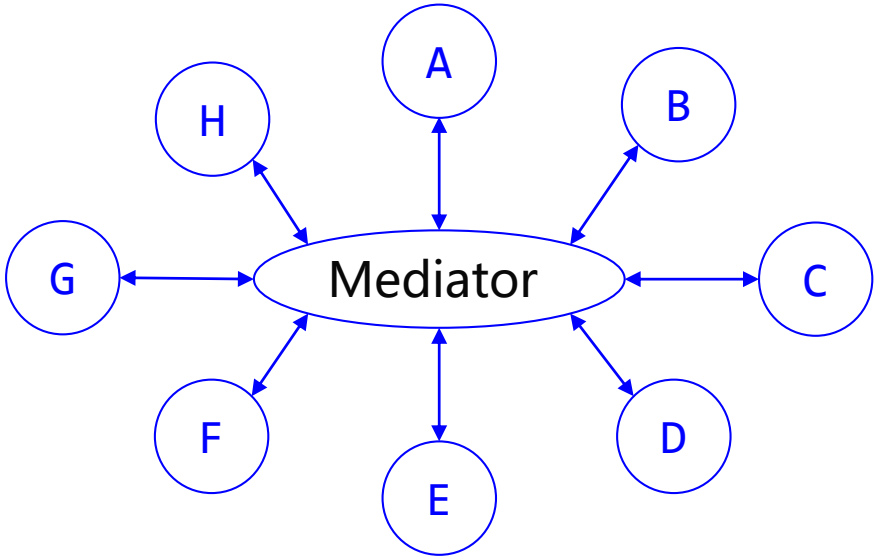
- 通常面向对象应用程序是由一组对象组成，这些对象为了提供服务而相互交互。



参与对象数目较少时，对象之间可以直接交互。即：点对点通讯。

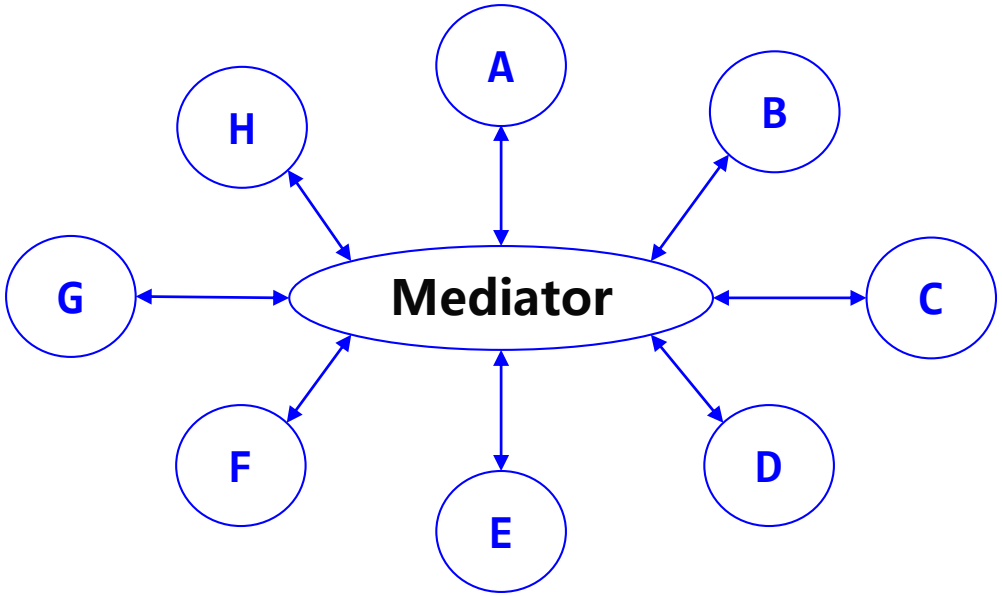


当参与对象越来越多时，这种点对点的通讯方式导致对象之间耦合度也越来越高



为了减少对象之间的紧密耦合，使用星形形式。这就是中介者模式逻辑图

6.3 中介模式的结构表达



中介者模式逻辑图

观察该逻辑图，发现有两种类型的对象：

- 中介者对象，应该抽象出类 Mediator
- 对象A~H都是参与者，应该抽象出不同具体类。根据面向对象中“碰到具体要抽象”的基本思路，因此还应该抽象出一个超类Colleague，形成参与者类层次结构

再分析两种类的关系。

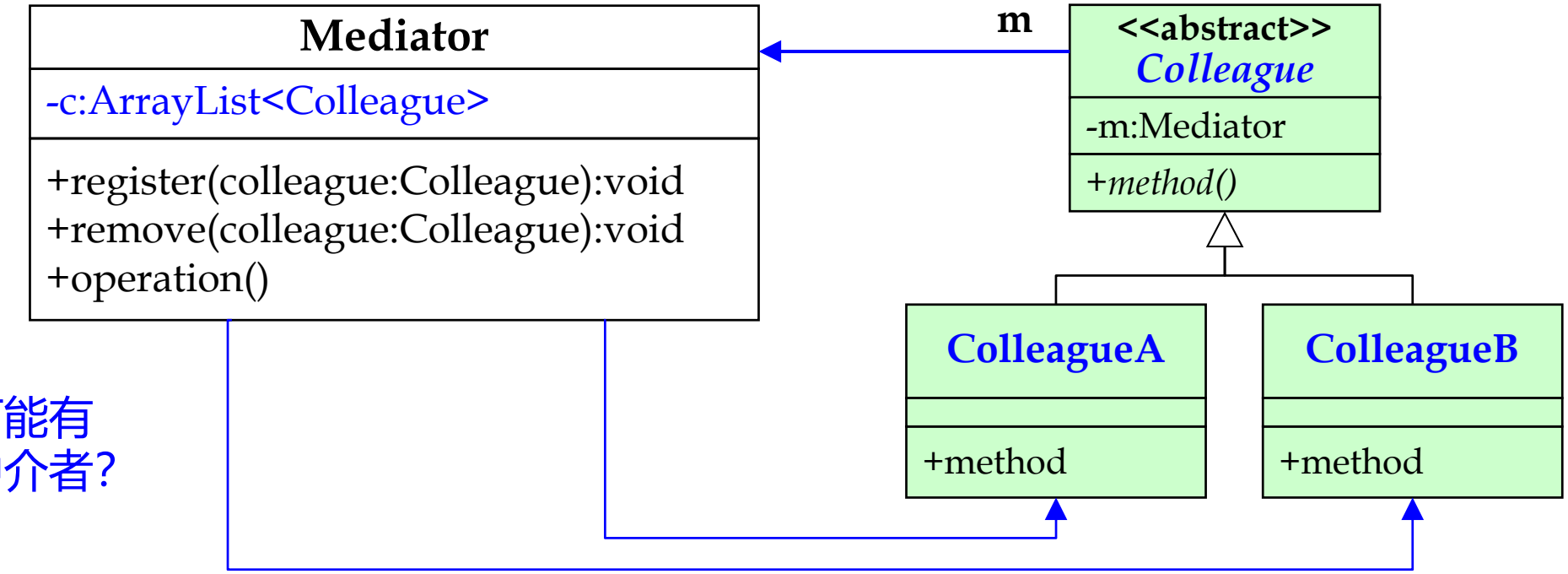
- 参与者类之间不应该有关联，点对点关联方式耦合度太高，这是我们设计成星型结构进行解耦的初衷。
- 所有参与者类应该和 Mediator 类之间有某种形式的关联：Mediator应该知道每一个参与者，每个参与者也应该知道Mediator，否则没有办法相互沟通。

最后，分析参与者类与中介者类中的属性和方法。

- Mediator 要知道所有的参与者，应该维护一个参与者对象的集合及其相关的操作
- 每个参与者都要知道Mediator对象，因此，可以从Colleague关联Mediator

6.3 中介模式的结构表达

中介类必须知道所有具体参与的同事，因此该类需要注册方法 register(), 用于让中介类保留所有参与对象到 ArrayList



有无可能有
多个中介者?

每个参与类都必须保持对中介类的引用，以便它可以调用中介类中的方法来做一些事情

本章总结

No	行为型模式	作用
1	观察者 Observer	维护多个观察者依赖，状态变化通知所有观察者
2	模板方法 Template Method	父类定义算法骨架，某些实现放在子类
3	策略 Strategy	每种算法独立封装，根据不同情况使用不同算法策略
4	职责链 Chain of Responsibility	所有处理者封装为链式结构，依次调用
5	状态 State	每种状态独立封装，不同状态内部封装了不同行为
6	中介者Mediator	取消类/对象的直接调用关系，使用中介者维护
7	命令 Command	将一个请求封装为一个对象，使发出请求的责任和执行请求的责任分割开
8	备忘录 Memento	把核心信息抽取出来，可以进行保存
9	解释器 Interpreter	定义语法解析规则
10	迭代器 Iterator	定义集合数据的遍历规则
11	访问者 Visitor	分离对象结构，与元素的执行算法

The end

December 10, 2025